

Resource Soundness with Precise Coeffect-Aware Semantics

JONATHAN CHAN, University of Pennsylvania, USA

STEPHANIE WEIRICH, University of Pennsylvania, USA

Coeffects provide a way of tracking the demands that programs make on their environments. Type-and-coeffect systems that describe how variables may be used gives insight on a program’s potential resource usage without running it, while coeffect-aware operational semantics make resource usage during evaluation explicit. One application of coeffects is to count how many times variables are used; in particular, *discardable* coeffects statically bound how many times *at most* they may be used. Soundness of type-and-coeffect systems with respect to the operational semantics then guarantees not only that a well-typed program evaluates safely, but also that static usage is an approximation of actual dynamic usage.

However, existing coeffect-aware semantics are not precise for discardable coeffects: they may claim that a particular program requires more resources than are actually used. As a result, we do not have a good model of what happens at run time. The type-and-coeffect systems we can show sound are then limited, because their precision is bounded by the coeffect-aware semantics. This is unfortunate because coeffects can be used to statically free unneeded resources by tracking what resources will not be used later in the computation.

We therefore develop the first precise coeffect-aware operational semantics and show soundness of a resource-tracking type-and-coeffect system with respect to this semantics. The culprit of imprecision is multiplication of coeffects: existing systems uniformly multiply the total resources required of a value by the number of usages of that value, even though different usages of a value may use varying amounts of its subcomponents. The strategy of our precise semantics is then to track coeffects on all subcomponents of a value during evaluation rather than merely on the variables that stand for those values.

1 Introduction

Type-and-coeffect systems annotate typing contexts Γ with *coeffects* q , typically per variable, forming contexts of the shape $x_1 :^{q_1} A_1, \dots, x_m :^{q_m} A_m$. Coeffect annotations, drawn from some partially-ordered semiring, enforce how the variables may be used. For example, for resource tracking, we use natural numbers as coeffects q . In a context containing $x :^q A$, the type system enforces that x can be accessed at most q times. Applications of type-and-coeffect systems include (bounded) linearity [Bernardy et al. 2018; Brunel et al. 2014; Ghica and Smith 2014; Petricek et al. 2014], (run-time and compile-time) irrelevance [Atkey 2018; Choudhury et al. 2021; McBride 2016; Moon et al. 2021], resource tracking [Orchard et al. 2019, and other work in the *Granule Project*], information flow control [Abel and Bernardy 2020; Gaboardi et al. 2016; Liepelt et al. 2026], and differential privacy [Gaboardi et al. 2016; Reed and Pierce 2010].

When designing type-and-coeffect systems, we would like them to be *sound*, *i.e.* they should provide an accurate abstraction of the execution of the program. Not only do we care about the usual type soundness, but we also want *coeffect soundness*: if a program is well typed under some coeffect-annotated context, then the program successfully evaluates only in ways described by its coeffects. For instance, if a program is well typed under $x :^q A$ as above, then we want to show that the program uses whatever is substituted for x no more than q times over the course of evaluation.

Therefore, to state what coeffect soundness means, we need a *coeffect-aware* operational semantics that dynamically tracks such usage during computation [Abel and Bernardy 2020; Bianchini et al. 2023; Choudhury et al. 2021; Torczon et al. 2024]. Instead of modelling substitution through β -reduction, a coeffect-aware semantics uses an environment or heap structure so that it may count the uses of each variable during the computation. With environments, uses are added together, providing a summary of the resources needed for the computation to run. In contrast, when tracking

resources with heaps, each usage of a variable decrements a current balance, leaving a residual at the end of the computation.

However, no existing work provides a precise accounting! Whether heap-based or environment-based, using small-step or big-step or an abstract machine, all of these semantics can overapproximate actual usage. It is possible for the operational semantics to say that a given computation used a value 5 times when it was only accessed 3 times.

These semantics are allowed to discard resources according to the semiring ordering. For quantities q with a discrete ordering, such as linear resources, the semantics can be precise because they are only related to themselves. The semantics of Torczon et al. [2024] and Bianchini et al. [2026] provide a precise accounting of nondiscardable resources.

But not all orderings on quantities are discrete. For example, we may wish to *bound* the uses of resources statically, counting them using natural numbers with their usual¹ ordering. Then the operational semantics may be imprecise and overreport the usage of some resources in a given run.

While this imprecision does not threaten soundness, it is troubling that we don't have an accurate record of what happens at run time. These approximations place a lower bound on the design of a type-and-coeffect system: it cannot check with more precision than the semantics can observe. Consequently, statically-tracked coeffects are approximations of approximations of dynamic usage.

Accurately tracking bounded resource usage in a computation is important. If, at a certain point in the computation, we know that the count of a resource is 0, meaning that it won't be used later, then the semantics may deallocate or release that resource immediately. For example, Choudhury et al. [2021] show that, by interpreting coeffect-annotated evaluation heaps as memory graphs, references with path weight 0 represent locations that can be garbage collected. Similarly, Torczon et al. [2024] include rules in the operational semantics that replace 0-usage values with "junk".

In this work, **we develop the first precise coeffect-aware semantics and show coeffect soundness for a resource-tracking type-and-coeffect system with respect to this semantics.** We say that a coeffect-aware operational semantics is precise if its coeffects account for all accesses of all values *and no more*. The key insight is observing the source of imprecision in the existing semantics. Coeffect annotations on variables are an approximation of how the *values* that the variables stand for are used. The existing semantics uniformly multiplies the available resources from a value by the quantity of the variable binding that value, but how many times a variable is used may differ from how many times its *value's subcomponents* are used, making this multiplication an imprecise representation of actual usage. Our strategy is therefore to track coeffects on all subcomponents of values used during evaluation rather than on each variable.

Because prior work on type-and-coeffect systems includes both call-by-value and call-by-name semantics, and because coeffect tracking is sensitive to evaluation order, we develop our new semantics in the context of call-by-push-value (CBPV) [Levy 2003], a low-level language that can host either semantics. Our work develops a novel precise coeffect instrumentation for CBPV, and our main result shows coeffect soundness of the resource-tracking type-and-coeffect system by Torczon et al. [2024] with respect to this precise semantics.

In addition to coeffect soundness, we also show the following properties of our precise semantics:

- *Approximation soundness.* If a term evaluates to some result under the imprecise resource-tracking semantics by Torczon et al. [2024], then it also evaluates to the same result using the precise semantics.

¹ By coeffect convention, $q_1 \leq q_2$ is a statement of restrictiveness: q_1 is *less* restrictive, while q_2 is *more* restrictive, which is why q_1 usages encompasses q_2 usages. Unfortunately, because 1 usage may be more than 0 usages (depending on one's notion of usage), we write $1 \leq 0$, which is the reverse of common *mathematical* convention.

- *Annotation irrelevance.* If coeffect instrumentation is erased from the precise semantics, it produces the same result as the uninstrumented semantics.
- *Annotation uniqueness.* There is a unique way of annotating coeffects on the values consumed during precise evaluation for a given annotated result.
- *Discarding.* If a value is known to be unused by the precise semantics, then it may be discarded by replacing it with a dummy value.

Our metatheoretic results are all mechanized in Rocq; the proof development can be found in our public repository [Chan and Weirich 2026]². The definitions and theorems in this paper are accompanied by references to the corresponding file and declaration, noted in square brackets.

The soundness of our precise semantics with respect to the approximate resource semantics is what allows us to show soundness with respect to the type-and-coeffect system. Annotation irrelevance and uniqueness tell us that, independent of typing, the precise annotations are the only correct way to annotate an evaluation without interfering with how evaluation proceeds. This means that it would be possible to augment the design of the type-and-coeffect system without having to redesign the precise semantics as well. Furthermore, the correctness of discarding unused values enables coeffect-driven resource deallocation.

2 Imprecision in Coeffect-Aware Semantics

To understand where imprecision in coeffect instrumentation can arise, consider the following program, which appends together the components of a pair of strings.³

$$\lambda p. \text{match } p \text{ as } (a, b) \Rightarrow \text{match } p \text{ as } (c, d) \Rightarrow a \mathbin{\dot{+}} d$$

We will consider its evaluation using a big-step environment-based semantics. This semantics describes the evaluation of terms M to their terminal configurations T using judgements of the form $\boxed{\rho \vdash M \Downarrow T}$. The environment ρ maps variables to their values, and in a coeffect-aware semantics (Section 2.1), is where variables are instrumented with their coeffects.

First, we show how this function evaluates when supplied with two strings, using a standard, non-coeffect-aware semantics. The subscript R on the turnstile below stands for a *raw*, uninstrumented semantics. We put the two strings in the environment and then supply them to the function in a pair. The result is their concatenation, as the judgement below demonstrates.

$$v \mapsto \text{"P"}, w \mapsto \text{"L"} \vdash_R (\lambda p. \text{match } p \text{ as } (a, b) \Rightarrow \text{match } p \text{ as } (c, d) \Rightarrow a \mathbin{\dot{+}} d) (v, w) \Downarrow \text{return "PL"} \quad [1]$$

To derive this judgement, the semantics must first evaluate the function argument, a pair of strings, by looking up the values of its components. This is the first premise required in the derivation of the judgement.

$$v \mapsto \text{"P"}, w \mapsto \text{"L"} \vdash_R (v, w) \Downarrow (\text{"P"}, \text{"L"}) \quad [2]$$

In the second premise of the derivation, we must evaluate the function application itself, which consists of deriving the evaluation of the function body with a new mapping from the function parameter p to its argument. Therefore, we must show the following.

$$p \mapsto (\text{"P"}, \text{"L"}) \vdash_R \text{match } p \text{ as } (a, b) \Rightarrow \text{match } p \text{ as } (c, d) \Rightarrow a \mathbin{\dot{+}} d \Downarrow \text{return "PL"} \quad [3]$$

(As v and w are no longer needed, we omit them from the environment to reduce visual clutter.)

Next, evaluating a match expression requires adding bindings for the components of the match to the environment. The components come from the pair that p maps to. Then to show the above, it suffices to derive the following evaluation. (For comparison with the instrumented semantics in

² <https://github.com/sweirich/precise-coeffects/tree/main/rocq>

³ While this program is awkward, it is a simple example of imprecision. Note that it could occur in practice from desugaring pair projections.

the next section, this time we keep the variables b and c around even though they won't be used.)

$$p \mapsto ("P", "L"), a \mapsto "P", b \mapsto "L" \vdash_R \text{match } p \text{ as } (c, d) \Rightarrow a \uplus d \Downarrow \text{return "PL"} \quad [4]$$

The second match expression evaluates similarly, now omitting the p no longer used.

$$a \mapsto "P", b \mapsto "L", c \mapsto "P", d \mapsto "L" \vdash_R a \uplus d \Downarrow \text{return "PL"} \quad [5]$$

Finally, the string append operation retrieves its arguments and returns its result.

$$a \mapsto "P", b \mapsto "L", c \mapsto "P", d \mapsto "L" \vdash_R "P" \uplus "L" \Downarrow \text{return "PL"} \quad [6]$$

As we assume that the append operation is primitive, we have a complete derivation.

2.1 Tracking Resource Usage with Coeffects

Now we annotate this derivation with coeffect tracking, following [Torczon et al. \[2024\]](#). This instrumented semantics isn't executable as it requires foreknowledge of resource usage to split the environment at certain steps in the evaluation. Instead, it is a declarative statement of resource usage, useful as a proof device to show coeffect soundness.

This instrumented semantics annotates variables in the environment with coeffects. Here, we use natural numbers for the coeffects, which count *resource usage*, or how many times a variable is used, and call these annotations *quantities*. Quantities also appear on function arguments, counting how many times the argument is used by the function.

In the instrumented semantics, the annotation of [Evaluation \[1\]](#) requires 2 uses each of v and w . This annotation is an overapproximation because these strings are each used only once by the append operation. (The v on the turnstile stands for evaluation with a vector of coeffects in the environment.)

$$v \mapsto^2 "P", w \mapsto^2 "L" \vdash_v (\lambda p. \text{match } p \text{ as } (a, b) \Rightarrow \text{match } p \text{ as } (c, d) \Rightarrow a \uplus d) (v, w)^2 \Downarrow \text{return "PL"}$$

Let's trace through this evaluation to see where the overapproximation comes from. Its derivation has two premises, one for the function argument and one for the function body, so the resources of the environment need to be split between them. First, evaluating a single pair requires 1 each of v and w . This is the annotation of the first premise, [Evaluation \[2\]](#).

$$v \mapsto^1 "P", w \mapsto^1 "L" \vdash_v (v, w) \Downarrow ("P", "L")$$

However, because the function demands 2 uses of its argument, we must multiply the resources in this evaluation by 2, requiring a total of $v \mapsto^2 "P", w \mapsto^2 "L"$. The second premise's portion of the environment is $v \mapsto^0 "P", w \mapsto^0 "L"$; this and the previous environment add together to produce the total usage in [Evaluation \[1\]](#). In the second premise [Evaluation \[3\]](#), we evaluate the function body with this environment extended with a new mapping $p \mapsto^2 ("P", "L")$.

$$v \mapsto^0 "P", w \mapsto^0 "L", p \mapsto^2 ("P", "L") \vdash_v \text{match } p \text{ as } (a, b) \Rightarrow \text{match } p \text{ as } (c, d) \Rightarrow a \uplus d \Downarrow \text{return "PL"}$$

In this environment, v and w are both marked 0, and cannot be used any further. Therefore, we will elide them from the example to reduce clutter.

Next, the first match expression splits the environment $p \mapsto^2 ("P", "L")$ between the 1 usage it consumes and the 1 left behind in the environment. It binds the pair components to a and b with quantity 1 each because they come from a 1-annotated mapping to the pair. This is the annotation of [Evaluation \[4\]](#).

$$p \mapsto^1 ("P", "L"), a \mapsto^1 "P", b \mapsto^1 "L" \vdash_v \text{match } p \text{ as } (c, d) \Rightarrow a \uplus d \Downarrow \text{return "PL"}$$

The second match expression does the same thing, using the final p and mapping c and d to its components, producing the annotation of [Evaluation \[5\]](#).

$$a \mapsto^1 "P", b \mapsto^1 "L", c \mapsto^1 "P", d \mapsto^1 "L" \vdash_v a \uplus d \Downarrow \text{return "PL"}$$

Finally, the string append operation uses 1 each of a and d , leaving behind 0 quantities of them in the environment (cf. [Evaluation \[6\]](#)).

$$a \mapsto^0 \text{"P"}, b \mapsto^1 \text{"L"}, c \mapsto^1 \text{"P"}, d \mapsto^0 \text{"L"} \vdash_v \text{"P"} \# \text{"L"} \Downarrow \text{return "PL"}$$

Notice that at the end of evaluating our program, b and c remain unused with 1 leftover quantity each. This is what we mean by the instrumentation being imprecise: we began with an environment containing 2 each of "P" and "L", but the evaluation in actuality only used 1 of each. A more precise semantics would have only demanded the initial environment $v \mapsto^1 \text{"P"}, w \mapsto^1 \text{"L"}$. All that the semantics can do now is to drop the unused resources via an explicit subsumption rule.

$$a \mapsto^0 \text{"P"}, b \mapsto^0 \text{"L"}, c \mapsto^0 \text{"P"}, d \mapsto^0 \text{"L"} \vdash_v \text{"P"} \# \text{"L"} \Downarrow \text{return "PL"}$$

The subsumption step is not optional—the semantics for primitive evaluation requires that the environment annotate all variables with 0. Crucially, to drop v and w , it must be that 1 is *discardable* in the semiring, meaning that we have $1 \leq 0$.

If the semiring order were discrete, meaning that all quantities are nondiscardable, then we would not be able to derive the instrumented evaluation as above. The imprecision in the evaluation is witnessed by subsumption, so this semantics is precise for nondiscardable quantities. For example, if we have a judgement that annotates a variable with a linear quantity, we know that it is used precisely one time during the computation. In contrast, a variable with an affine quantity does not indicate whether the evaluation used the resource zero or one time, even though we have the complete evaluation.

In other words, while this semantics is precise for nondiscardable quantities, it is approximate for discardable quantities. In contrast, our novel precise semantics, which we demonstrate next, is always precise regardless of discardability.

2.2 Precise Coeffect-Aware Semantics

The existing vector semantics was imprecise because of the assumption that the usages of the subvalues of what a variable maps to in the environment scale with the usages of the variable itself. For our examples above, the incorrect assumption is that both "P" and "L" are used twice when $p \mapsto^2 (\text{"P"}, \text{"L"})$ is used twice.

To resolve this problem, instead of annotating variables with quantities, we annotate every *subvalue* with a quantity. When using a variable from the environment, the quantities in the value it maps to may be split asymmetrically between the usage and the remaining environment.

Revisiting our example [Evaluation \[1\]](#), we annotate the initial environment to say that we only require 1 use each of the strings "P" and "L". Note that the variables v and w are not annotated with quantities; it is their values that carry coeffect annotations.

As before, the pair argument is annotated with 2 because it will be matched on 2 times during the evaluation. The result of evaluation is a string "PL" that can be used once. (The p on the turnstile stands for evaluation with a precise environment.)

$$v \mapsto \text{"P"}^1, w \mapsto \text{"L"}^1 \vdash_p (\lambda p. \text{match } p \text{ as } (a, b) \Rightarrow \text{match } p \text{ as } (c, d) \Rightarrow a \# d) (v, w)^2 \Downarrow \text{return "PL"}^1$$

We again split the environment between the first and second premises, but this time the quantities on the values themselves are split apart such that they pointwise add to the original quantities. We have $v \mapsto \text{"P"}^1, w \mapsto \text{"L"}^1$ for the first premise evaluating the function argument, and $v \mapsto \text{"P"}^0, w \mapsto \text{"L"}^0$ for the second premise evaluating the function body. The former is the annotation of [Evaluation \[2\]](#), shown below. Here we can put strings annotated by 1 inside a tuple annotated by 2; this semantics tracks the parts individually.

$$v \mapsto \text{"P"}^1, w \mapsto \text{"L"}^1 \vdash_p (v, w) \Downarrow (\text{"P"}^1, \text{"L"}^1)^2$$

$$\begin{array}{ll}
q ::= 0 \mid 1 \mid q_1 + q_2 \mid q_1 \cdot q_2 \mid \gamma(x) & A ::= \top \mid A_1 + A_2 \mid A_1 \times A_2 \mid \cup B \\
\gamma ::= \bar{0} \mid \bar{1} \mid \gamma_1 + \gamma_2 \mid q \cdot \gamma & B ::= F^q A \mid A^q \rightarrow B \mid B_1 \& B_2 \\
V ::= x \mid () \mid \text{inl } V \mid \text{inr } V \mid (V_1, V_2) \mid \{M\} & \gamma \cdot \Gamma ::= \emptyset \mid \gamma \cdot \Gamma, x :^q A \\
M ::= V! \mid \lambda x. M \mid M V^q \mid \text{return}^q V \mid \text{let } x \leftarrow M_1 \text{ in } M_2 \mid \langle M_1, M_2 \rangle \mid \text{fst } M \mid \text{snd } M \\
& \mid V^q ; M \mid \text{match } V^q \text{ as } (x_1, x_2) \Rightarrow M \mid \text{case } V^q \text{ of } \text{inl } x_1 \Rightarrow M_1 \mid \text{inr } x_2 \Rightarrow M_2
\end{array}$$

Fig. 1: Syntax of CBPV with coefficients [cbpv.syntax: Val, Comp, ValTy, CompTy]

In the second premise evaluating the function body, p maps to this resulting pair where the top-level and subcomponents are separately annotated. We again omit its portion of the environment that is no longer used, namely v and w . The following is the annotation of [Evaluation](#) [3].

$$p \mapsto ("P^1", "L^1")^2 \vdash_P \text{match } p \text{ as } (a, b) \Rightarrow \text{match } p \text{ as } (c, d) \Rightarrow a \uparrow\uparrow d \Downarrow \text{return "PL"}^1$$

Now is when these subvalue annotations come into play: we split the environment into two asymmetric portions, $p \mapsto ("P^1", "L^0")^1$ and $p \mapsto ("P^0", "L^1")^1$. The first match expression retrieves the former portion of p , mapping a and b to its components as annotated, and leaves behind the latter piece (cf. [Evaluation](#) [4]).

$$p \mapsto ("P^0", "L^1")^1, a \mapsto "P^1", b \mapsto "L^0" \vdash_P \text{match } p \text{ as } (c, d) \Rightarrow a \uparrow\uparrow d \Downarrow \text{return "PL"}^1$$

The second match expression splits the environment into $p \mapsto ("P^0", "L^1")^1$ and the remaining environment now containing $p \mapsto ("P^0", "L^0")^0$. Then c and d map to the components similarly (cf. [Evaluation](#) [5]), omitting the p that is no longer used.

$$a \mapsto "P^1", b \mapsto "L^0", c \mapsto "P^0", d \mapsto "L^1" \vdash_P a \uparrow\uparrow d \Downarrow \text{return "PL"}^1$$

Finally, the string append operation uses the final resources in a and d (cf. [Evaluation](#) [6]). As all values in the environment are annotated with 0, including the mappings $v \mapsto "P^0", w \mapsto "L^0, p \mapsto ("P^0", "L^0")^0$ we omitted earlier, no subsumption step is required before the primitive append operation.

$$a \mapsto "P^0", b \mapsto "L^0", c \mapsto "P^0", d \mapsto "L^0" \vdash_P "P^1" \uparrow\uparrow "L^1" \Downarrow \text{return "PL"}^1$$

The precise semantics has accounted for all resources, no less and no more, because subsumption was not required at any point.

The source of imprecision in the vector semantics is that, although the semantics knows how many times the function uses its argument, it has to treat all uses of the (sub-)values uniformly. The rules of the semantics then use multiplication to account for multiple uses of a variable. The key mechanism in the precise semantics is the ability to split the resources tracked in the environment within the value, not just at the variable mapping. As a result, the precise semantics never needs to multiply to calculate aggregate usage.

3 CBPV with Coeffects

The language for which we provide precise semantics is call-by-push-value with coeffects. Its syntax is given in [Figure 1](#) and consists of the usual CBPV constructs, divided between values and computations, which are then annotated by coeffects. Values consist of unit $()$ in the unit type $\top$, eliminated by the sequencing $;$ computation; pairs (V_1, V_2) in the pair type $A_1 \times A_2$, eliminated by pattern matching; sums of $\text{inl } V$ and $\text{inr } V$ injections in the sum type $A_1 + A_2$, eliminated by case analysis; and thunks $\{M\}$ in the thunk type $\cup B$, which are suspended computations resumed by forcing $\{M\}!$. Computations, aside from the elimination forms already mentioned, consist of returning values in the type $F^q A$, which get let-bound in a computation; functions $\lambda x. M$ in the

function type $A^q \rightarrow B$, which get applied to values $M V^q$; and products $\langle M_1, M_2 \rangle$ in the product type $B_1 \& B_2$, of which `fst` and `snd` projections may be taken. Because computation products are unremarkable in terms of coeffect tracking, we omit them from further discussion in this paper, though they are included in our mechanization. Omitted rules may be found in [Appendix A](#).

Coeffects consist of quantities q drawn from a *preordered semiring*, with an addition operation $+$, an additive identity 0 , a multiplication operation $\cdot$, a multiplicative identity 1 , and an ordering relation $\leq$. The usual associativity, commutativity, identity, annihilation, distributivity, reflexivity, transitivity, antisymmetry, and monotonicity properties are listed in [Figure 2](#). Some of these are derivable from other axioms, such as right additive identity from the left with commutativity; they are listed together anyway for clarity.

$$\begin{array}{llll}
(q_1 + q_2) + q_3 = q_1 + (q_2 + q_3) & q + 0 = q & 0 + q = q & q_1 + q_2 = q_2 + q_1 \\
(q_1 \cdot q_2) \cdot q_3 = q_1 \cdot (q_2 \cdot q_3) & q \cdot 1 = q & 1 \cdot q = q & q \cdot 0 = 0 \quad 0 \cdot q = 0 \\
q \cdot (q_1 + q_2) = q \cdot q_1 + q \cdot q_2 & (q_1 + q_2) \cdot q = q_1 \cdot q + q_2 \cdot q & & \\
q \leq q & q_1 \leq q_2 \leq q_3 \Rightarrow q_1 \leq q_3 & & \\
q_1 \leq q_2 \Rightarrow q + q_1 \leq q + q_2 & q_1 \leq q_2 \Rightarrow q_1 + q \leq q_2 + q & & \\
q_1 \leq q_2 \Rightarrow q \cdot q_1 \leq q \cdot q_2 & q_1 \leq q_2 \Rightarrow q_1 \cdot q \leq q_2 \cdot q & &
\end{array}$$

Fig. 2: Properties of preordered semirings [`common.coeffects`:POSr]

By coeffect convention, the order notation is the reverse of usual mathematical order, e.g. for the naturals and their natural total order, we have $\dots \leq 2 \leq 1 \leq 0$. Nonetheless, to avoid further confusion, we say that 2 is “more”/“larger”/“greater”/etc. than 1 despite the order notation.

Atop a preordered semiring, we require a few more axioms and impose that quantities are:

- decidable, i.e. $q_1 = q_2 \vee q_1 \neq q_2$ for any q_1, q_2 ;
- nontrivial, i.e. $0 \neq 1$;
- nonnegative, i.e. $0 \leq q \Rightarrow q = 0$;
- zerosumfree, i.e. $q_1 + q_2 = 0 \Rightarrow q_1 = 0 \wedge q_2 = 0$; and
- entire, i.e. $q_1 \cdot q_2 = 0 \Rightarrow q_1 = 0 \vee q_2 = 0$.

Decidability is required because there are rules in the upcoming semantics which branch on $q = 0$. Nontriviality ensures that we can distinguish a usage from a non-usage. The semiring needs to be nontrivial, nonnegative, zerosumfree, and entire to prove the lemmas we need for type safety in [Section 6.3](#), just as in prior work by [Torczon et al. \[2024\]](#).

Throughout the examples in this paper, we use the naturals as our representative ordered semiring, interpreting quantities as the number of times a variable may be used. Here, the order is a notion of *restrictiveness*: a 0 -quantity variable is the most restrictive and its value may not be used at all, while the larger the quantity, the less restrictive the usage is. If $q_1 \leq q_2$, then q_1 is less restrictive than q_2 , since a q_1 -quantity variable may be used q_1 times, or it may be used q_2 times. Note that we don’t require $1 \leq 0$ (equivalent to discardability, or $q \leq 0$ for all q), and that the order may contain only some discardable quantities, or be entirely discrete (i.e. defined as equality). In a discrete order, a q -quantity variable must be used precisely q times.

Although much of this paper is phrased in terms of counting for intuitive understanding, counting is not the only concrete interpretation of quantities. Only having quantities 0 and 1 is already useful information in tracking whether a variable is used or not. Knowing that a variable is unused means that, as we do in our operational semantics in [Sections 4](#) and [5](#), values bound to that variable need not be evaluated. This application requires that $0 \not\leq q + 1$, which [Choudhury et al. \[2021\]](#) call *zero-unusable* and follows from the nonnegative and zerosumfree properties.

With the quantities $0, 1, \omega$ such that $\omega + 1 = \omega$, where $\omega \leq 1$ and $\omega \leq 0$ but not $1 \leq 0$, we have the none-one-tens system by McBride [2016], representing no usage, linear usage, or arbitrary usage, respectively. If additionally $1 \leq 0$, then a 1 -quantity variable may be used affinely, *i.e.* at most once, but possibly never.

Another useful ordered semiring consisting of elements $1 = L \leq M \leq H = 0$ represents low, medium, and high security levels in information flow analysis. The order similarly represents restrictiveness, as a L -security variable is the least restrictive, while a H -security variable is the most restrictive. The value of a M - or H -security variable, for instance, may not be inspected in a L -security setting, which would otherwise represent an information leak. Liepelt et al. [2026] show that to prove a noninterference theorem for a graded coeffect system, these security levels need to further satisfy *lax idempotence* $q \cdot q \leq q$, forming what they call a *noninterfering ring*.

To assign quantities to a list of variables, we have vectors⁴ γ of mappings from variables to quantities, which we also call *resource vectors*. Quantity operations are extended to pointwise vector addition $\gamma_1 + \gamma_2$, where $(\gamma_1 + \gamma_2)(x) = \gamma_1(x) + \gamma_2(x)$, and scalar vector multiplication $q \cdot \gamma$, where $(q \cdot \gamma)(x) = q \cdot \gamma(x)$, and the preorder is extended to a pointwise vector preorder, where $\gamma_1 \leq \gamma_2$ when for all x , $\gamma_1(x) \leq \gamma_2(x)$. A bar over a quantity is shorthand for a resource vector containing only that quantity: $\bar{0}$ is a vector of 0 s, $\bar{1}$ is a vector of 1 s, and $\bar{q}$ is a vector of q s. The notation for resource vector extension is combined with typing context extension so that $\gamma \cdot \Gamma, x : ^q A$ extends the resource vector by q and the typing context by A simultaneously.

3.1 A Type-and-Coeffect System for CBPV

The typing rules for CBPV with coeffects, taken from Torczon et al. [2024], are given in Figure 3. Ignoring all quantities q and resource vectors γ , the rules are standard for CBPV with unit, value sums, value pairs, and computation products. The coeffects then track usage of variables from the typing context by annotating each of those variables with quantities. Rule **T-UNIT** uses no variables, so its resources are $\bar{0}$; otherwise, in rules **T-THUNK**, **T-PAIR**, **T-INL**, **T-INR**, **T-FORCE**, and **T-SEQ**, the resources required are the sum of those required for their premises.

To use a variable once, it must be annotated with 1 , and all other resources in rule **T-VAR** must be $\bar{0}$ as they are not used. The computation $\text{return}^q V$ means that V may be used q times, so rule **T-RET** multiplies the resources required of V by q . As an example, $x : ^2 T \vdash \text{return}^2 x : F^2 T$ requires 2 of x because two unit values need to be available for use.

When type checking the let body in rule **T-LET** binding the value of some $F^q A$ to x , there are q of x available for use. However, this doesn't mean the resources required of the bound computation need to be multiplied by q , as the quantity is already accounted for in its typing derivation. For example, the following derivation of a let-bound returned variable demonstrates that the accounting is handled by rule **T-RET**.

$$\begin{array}{c}
 \text{T-VAR} \frac{}{x : ^1 T \vdash x : T} \\
 \text{T-RET} \frac{x : ^2 T \vdash \text{return}^2 x : F^2 T \quad \dots \quad x : ^0 T, y : ^2 T \vdash M : B}{x : ^2 T \vdash \text{let } y \leftarrow \text{return}^2 x \text{ in } M : B} \text{T-LET}
 \end{array}$$

Similarly to let bindings, functions abstract over a variable of quantity q in rule **T-ABS**, which is recorded in its function type. The difference is that quantity multiplication only occurs during application in rule **T-APP**, since the resources required by the function argument aren't known until the function is actually applied to the argument. The quantity recorded on the function type indicates what the argument resources must be multiplied by.

⁴ Mathematically, they form elements of not a vector space but a *preordered left q -semimodule* [Golan 2003]. Their abstract properties aren't important because we work with the concrete instance of finite lists of quantities.

$\boxed{\gamma \cdot \Gamma \vdash V : A} \quad \boxed{\gamma \cdot \Gamma \vdash M : B}$	(value and computation typing)	
T-VAR	T-UNIT	T-INL
$\frac{}{\bar{0} \cdot \Gamma_1, x : ^1A, \bar{0} \cdot \Gamma_2 \vdash x : A}$	$\frac{}{\bar{0} \cdot \Gamma \vdash () : \top}$	$\frac{\gamma \cdot \Gamma \vdash V : A_1}{\gamma \cdot \Gamma \vdash \text{inl } V : A_1 + A_2}$
T-PAIR	T-THUNK	T-VSUB
$\frac{\gamma_1 \cdot \Gamma \vdash V_1 : A_1 \quad \gamma_2 \cdot \Gamma \vdash V_2 : A_2}{(\gamma_1 + \gamma_2) \cdot \Gamma \vdash (V_1, V_2) : A_1 \times A_2}$	$\frac{\gamma \cdot \Gamma \vdash M : B}{\gamma \cdot \Gamma \vdash \{M\} : \cup B}$	$\frac{\gamma_1 \leq \gamma_2 \quad \gamma_2 \cdot \Gamma \vdash V : A}{\gamma_1 \cdot \Gamma \vdash V : A}$
T-ABS	T-RET	T-SEQ
$\frac{\gamma \cdot \Gamma, x : ^qA \vdash M : B}{\gamma \cdot \Gamma \vdash \lambda x. M : A^q \rightarrow B}$	$\frac{\gamma \cdot \Gamma \vdash V : A}{q \cdot \gamma \cdot \Gamma \vdash \text{return}^q V : F^q A}$	$\frac{\gamma_1 \cdot \Gamma \vdash V : \top \quad \gamma_2 \cdot \Gamma \vdash M : B}{(q \cdot \gamma_1 + \gamma_2) \cdot \Gamma \vdash V^q ; M : B}$
T-APP	T-LET	T-FORCE
$\frac{\gamma_1 \cdot \Gamma \vdash M : A^q \rightarrow B \quad \gamma_2 \cdot \Gamma \vdash V : A}{(\gamma_1 + q \cdot \gamma_2) \cdot \Gamma \vdash M V^q : B}$	$\frac{\gamma_1 \cdot \Gamma \vdash M_1 : F^q A \quad \gamma_2 \cdot \Gamma, x : ^qA \vdash M_2 : B}{(\gamma_1 + \gamma_2) \cdot \Gamma \vdash \text{let } x \leftarrow M_1 \text{ in } M_2 : B}$	$\frac{\gamma \cdot \Gamma \vdash V : \cup B}{\gamma \cdot \Gamma \vdash V! : B}$
T-CASE		
$\frac{q \leq 1 \quad \gamma_1 \cdot \Gamma \vdash V : A_1 + A_2 \quad \gamma_2 \cdot \Gamma, x_1 : ^qA_1 \vdash M_1 : B \quad \gamma_2 \cdot \Gamma, x_2 : ^qA_2 \vdash M_2 : B}{(q \cdot \gamma_1 + \gamma_2) \cdot \Gamma \vdash \text{case } V^q \text{ of } \text{inl } x_1 \Rightarrow M_1 \mid \text{inr } x_2 \Rightarrow M_2 : B}$		
T-MATCH		T-CSUB
$\frac{\gamma_1 \cdot \Gamma \vdash V : A_1 \times A_2 \quad \gamma_2 \cdot \Gamma, x_1 : ^qA_1, x_2 : ^qA_2 \vdash M : B}{(q \cdot \gamma_1 + \gamma_2) \cdot \Gamma \vdash \text{match } V^q \text{ as } (x_1, x_2) \Rightarrow M : B}$		$\frac{\gamma_1 \leq \gamma_2 \quad \gamma_2 \cdot \Gamma \vdash M : B}{\gamma_1 \cdot \Gamma \vdash M : B}$

Fig. 3: Typing rules for CBPV with coeffects (excerpt) [cbpv.typing:VWt,CWt]

Generally, the elimination forms for values permit implicit multiple eliminations. For example, Rule T-SEQ eliminates q unit values; we can think of $V^2 ; M$, for instance, as being equivalent to $V^1 ; V^1 ; M$. Multiple eliminations require multiplying the resources required for the value by q , but make q copies of each subterm available in pattern matching. Eliminating value pairs in rule T-MATCH binds each component in the subsequent computation; if we have q of a pair, then its components may each be used q times. Eliminating injections in rule T-CASE binds an injected value in each branch of computation; if we have q of an injection, then the injected value may be used q times in either branch.

Case expressions additionally require that there is at least one injection being eliminated to know whether it is a left or right injection. If there are no injections to branch on, then there is no way to know which branch to take. In contrast, matching on no pairs is permitted if its components are used no times, so they have no quantity restrictions.

Rules T-VSUB and T-CSUB are needed to weaken the amount of resources required for a value or a computation from fewer resources to more resources. For instance, given $x : ^1A \vdash x : A$ by rule T-VAR, we can derive $x : ^2A \vdash x : A$, and given $x : ^2A \vdash \text{return}^2 x : F^2 A$ by rule T-RET, we can derive $x : ^3A \vdash \text{return}^2 x : F^2 A$. Weakening resources is necessary because the precise amount

needed can't always be known statically due to branching computations. For example,

$$x : ^2\top + \top \vdash \text{case } x^2 \text{ of } \text{inl } y \Rightarrow y^2 ; M \mid \text{inr } z \Rightarrow z^1 ; M : B$$

uses 2 of the left injection y and 1 of the right injection z , but because we don't know whether x is a left or right injection, we need to provide adequate resources for both branches, which means approximating the right branch with $z : ^2\top$.

4 Imprecise Resource Vector Semantics

Figure 4 presents the imprecise environment-based big-step semantics for CBPV, instrumented with a resource vector to track the required quantities of each value in the environment. We therefore refer to it as the (resource) vector semantics. The way resources are added and multiplied in evaluation rules mirrors their corresponding typing rules. These rules are adapted from the resource semantics by Torczon et al. [2024].

The judgement $[\gamma \cdot \rho \vdash_v V \Downarrow W]$ evaluates a value V under environment ρ with resources γ and yields a closed value W . This process resolves all variable references to their definitions in the environment and saves the current resource vector and environment in the closures of thunks. Closed values include an additional *junk value* ζ , representing a value in the environment that will never be used; consequently, junk values have no evaluation rule.

The judgement $[\gamma \cdot \rho \vdash_v M \Downarrow T]$ evaluates a computation M and yields a terminal computation T , which is either a returned closed value, or a residual configuration, i.e. an abstraction paired with a resource vector and environment so that computation may continue.

For example, consider the evaluation $x \mapsto^1 W \vdash_v x!()^1 \Downarrow \text{return}^1((), ())$ in an environment where x is a thunked function, $W = \{y \mapsto^1 () ; \lambda z. \text{return}^1(y, z)\}$. First, we evaluate $x!$ to a terminal, a new environment paired with a function.

$$\frac{x \mapsto^1 W \vdash_v x \Downarrow W \quad \text{Ev-Var} \quad \frac{y \mapsto^1 () \vdash_v \lambda z. \text{return}^1(y, z) \Downarrow y \mapsto^1 () ; \lambda z. \text{return}^1(y, z)}{\text{Ev-Abs}}}{x \mapsto^1 W \vdash_v x! \Downarrow y \mapsto^1 () ; \lambda z. \text{return}^1(y, z)} \text{Ev-Force}$$

While forcing a thunk uses its saved environment (which includes y , not x), evaluating a function propagates the current environment to the terminal as the body of the function has not yet been evaluated. This derivation is the first premise (1st) to the use of rule **Ev-App** below. The second premise simply evaluates the unit value.

In the third premise of rule **Ev-App**, the function body evaluates under that propagated environment extended by the function argument z . If the environment weren't propagated, we would have lost the mapping for y .

$$\frac{1 \neq 0 \quad (1^{\text{st}}) \quad \emptyset \vdash_v () \Downarrow () \quad y \mapsto^1 (), z \mapsto^1 () \vdash_v \text{return}^1(y, z) \Downarrow \text{return}^1((), ())}{x \mapsto^1 W \vdash_v x!()^1 \Downarrow \text{return}^1((), ())} \text{Ev-App}$$

The condition $1 \neq 0$ comes from the quantity on the function argument, which prevents values from the environment from being used when they appear in function arguments that are not used. As a counterexample, consider the following invalid application to a 0-quantity argument.

$$\frac{0 \neq 0 \quad x \mapsto^0 W \vdash_v \lambda y. \text{return}^1() \Downarrow x \mapsto^0 W ; \lambda y. \text{return}^1() \quad x \mapsto^1 W \vdash_v x \Downarrow W \quad x \mapsto^0 W, y \mapsto^0 W \vdash_v \text{return}^1() \Downarrow \text{return}^1()}{x \mapsto^{0+0+1} W \vdash_v (\lambda y. \text{return}^1()) x^0 \Downarrow \text{return}^1()} \text{Ev-App}$$

If this derivation were valid via omitting the quantity restriction, then it would state that this evaluation requires $0 + 0 \cdot 1 = 0$ uses of x despite evaluating it in the second premise. Therefore, following Torczon et al. [2024], we introduce *junk rules*, and the evaluation of function application depends on whether we actually require resources ($q \neq 0$) or not ($q = 0$). The condition on q

$W ::= () \mid \text{inl } W \mid \text{inr } W \mid (W_1, W_2) \mid \{\gamma \cdot \rho; M\} \mid \ddagger$			
$T ::= \text{return}^q W \mid \gamma \cdot \rho; \lambda x. M$			
$\gamma \cdot \rho ::= \emptyset \mid \gamma \cdot \rho, x \mapsto^q W$			
$\boxed{\gamma \cdot \rho \vdash_v V \Downarrow W}$		$\boxed{\gamma \cdot \rho \vdash_v M \Downarrow T}$	
(vector-instrumented evaluation)			
EV-VAR	EV-UNIT	EV-THUNK	
$\frac{}{\bar{0} \cdot \rho_1, x \mapsto^1 W, \bar{0} \cdot \rho_2 \vdash_v x \Downarrow W}$	$\frac{}{\bar{0} \cdot \rho \vdash_v () \Downarrow ()}$	$\frac{}{\gamma \cdot \rho \vdash_v \{M\} \Downarrow \{\gamma \cdot \rho; M\}}$	
EV-INL	EV-PAIR	EV-VSUB	
$\frac{\gamma \cdot \rho \vdash_v V \Downarrow W}{\gamma \cdot \rho \vdash_v \text{inl } V \Downarrow \text{inl } W}$	$\frac{\gamma_1 \cdot \rho \vdash_v V_1 \Downarrow W_1 \quad \gamma_2 \cdot \rho \vdash_v V_2 \Downarrow W_2}{(\gamma_1 + \gamma_2) \cdot \rho \vdash_v (V_1, V_2) \Downarrow (W_1, W_2)}$	$\frac{\gamma_1 \leq \gamma_2 \quad \gamma_2 \cdot \rho \vdash_v V \Downarrow W}{\gamma_1 \cdot \rho \vdash_v V \Downarrow W}$	
EV-FORCE	EV-ABS	EV-SEQ	
$\frac{\gamma \cdot \rho \vdash_v V \Downarrow \{\gamma' \cdot \rho'; M\} \quad \gamma' \cdot \rho' \vdash_v M \Downarrow T}{\gamma \cdot \rho \vdash_v V! \Downarrow T}$	$\frac{}{\gamma \cdot \rho \vdash_v \lambda x. M \Downarrow \gamma \cdot \rho; \lambda x. M}$	$\frac{q \neq 0 \quad \gamma_1 \cdot \rho \vdash_v V \Downarrow () \quad \gamma_2 \cdot \rho \vdash_v M \Downarrow T}{(q \cdot \gamma_1 + \gamma_2) \cdot \rho \vdash_v V^q; M \Downarrow T}$	
EV-LET	EV-RET		
$\frac{\gamma_1 \cdot \rho \vdash_v M_1 \Downarrow \text{return}^q W \quad \gamma_2 \cdot \rho, x \mapsto^q W \vdash_v M_2 \Downarrow T}{(\gamma_1 + \gamma_2) \cdot \rho \vdash_v \text{let } x \leftarrow M_1 \text{ in } M_2 \Downarrow T}$	$\frac{q \neq 0 \quad \gamma \cdot \rho \vdash_v V \Downarrow W}{q \cdot \gamma \cdot \rho \vdash_v \text{return}^q V \Downarrow \text{return}^q W}$		
EV-APP	EV-MATCH		
$\frac{q \neq 0 \quad \gamma_1 \cdot \rho \vdash_v M \Downarrow \gamma' \cdot \rho'; \lambda x. M' \quad \gamma_2 \cdot \rho \vdash_v V \Downarrow W \quad \gamma' \cdot \rho', x \mapsto^q W \vdash_v M' \Downarrow T}{(\gamma_1 + q \cdot \gamma_2) \cdot \rho \vdash_v M V^q \Downarrow T}$	$\frac{q \neq 0 \quad \gamma_1 \cdot \rho \vdash_v V \Downarrow (W_1, W_2) \quad \gamma_2 \cdot \rho, x_1 \mapsto^q W_1, x_2 \mapsto^q W_2 \vdash_v M \Downarrow T}{(q \cdot \gamma_1 + \gamma_2) \cdot \rho \vdash_v \text{match } V^q \text{ as } (x_1, x_2) \Rightarrow M \Downarrow T}$		
EV-CASE-INL	EV-CSUB		
$\frac{q \leq 1 \quad \gamma_1 \cdot \rho \vdash_v V \Downarrow \text{inl } W \quad \gamma_2 \cdot \rho, x_1 \mapsto^q W \vdash_v M_1 \Downarrow T}{(q \cdot \gamma_1 + \gamma_2) \cdot \rho \vdash_v \text{case } V^q \text{ of inl } x_1 \Rightarrow M_1 \mid \text{inr } x_2 \Rightarrow M_2 \Downarrow T}$	$\frac{\gamma_1 \leq \gamma_2 \quad \gamma_2 \cdot \rho \vdash_v M \Downarrow T}{\gamma_1 \cdot \rho \vdash_v M \Downarrow T}$		
EV-RET-JUNK	EV-SEQ-JUNK		
$\frac{q = 0}{q \cdot \gamma \cdot \rho \vdash_v \text{return}^q V \Downarrow \text{return}^q \ddagger}$	$\frac{q = 0 \quad \gamma \cdot \rho \vdash_v M \Downarrow T}{\gamma \cdot \rho \vdash_v V^q; M \Downarrow T}$		
EV-APP-JUNK	EV-MATCH-JUNK		
$\frac{q = 0 \quad \gamma \cdot \rho \vdash_v M \Downarrow \gamma' \cdot \rho'; \lambda x. M' \quad \gamma' \cdot \rho', x \mapsto^q \ddagger \vdash_v M' \Downarrow T}{\gamma \cdot \rho \vdash_v M V^q \Downarrow T}$	$\frac{q = 0 \quad \gamma \cdot \rho, x_1 \mapsto^q \ddagger, x_2 \mapsto^q \ddagger \vdash_v M \Downarrow T}{\gamma \cdot \rho \vdash_v \text{match } V^q \text{ as } (x_1, x_2) \Rightarrow M \Downarrow T}$		

Fig. 4: Big-step resource vector semantics (excerpt) [cbpv.vector.semantics:EvalVal, EvalComp]

determines whether the argument is evaluated in rule **EV-APP** or not in the junk rule **EV-APP-JUNK**. If not, then x gets bound to the junk value $\bot$ in the environment when evaluating the body, which is safe as the argument is unused. Junk rules are justified by **Corollary 6.20**, which states that substituting $\bot$ for 0-quantity variables during evaluation is safe.

In every evaluation where a resource vector is multiplied by some q , there may be a corresponding junk rule for $q = 0$. In rule **EV-RET**, if we return $q \neq 0$ of a value, we must evaluate that value; otherwise, we return $\bot$ in rule **EV-RET-JUNK**. Similarly, in rule **EV-MATCH**, the value is evaluated to a pair if $q \neq 0$ of its components are needed, and otherwise in rule **EV-MATCH-JUNK**, those components are filled in by $\bot$ in the match body. In rule **EV-SEQ**, the value is evaluated to a unit if $q \neq 0$ units are used, and otherwise in rule **EV-SEQ-JUNK**, evaluating the value is skipped. Case expressions are the exception, because we must always evaluate the scrutinee of an injection, as discussed in **Section 3**. Hence, there are no junk rules for rules **EV-CASE-INL** and **EV-CASE-INR**.

For both value and computation evaluation, there are subsumption rules **EV-VSUB** and **EV-CSUB**, but for a different reason than those in the typing rules. During static typing, subsumption is required to approximate the resources used by branching computations, since which branch is taken is not yet known. During evaluation, subsumption is instead necessary to drop variables with discardable quantities, as we saw in **Section 2**.

4.1 Imprecision in Resource Vector Semantics

The resource vector semantics is imprecise when more quantities are demanded than are actually used. Imprecision arises wherever a scalar multiplication of resources occurs while eliminating composite values, since those composite values may use resources in different ways. This means that rules **EV-APP**, **EV-MATCH**, **EV-INL**, and **EV-INR** all have the potential to be imprecise, involving the composite pair, injection, and thunk values. (Units have nothing in them.) The presence of imprecision is indicated by the need to weaken the environment using rule **EV-CSUB**.

For rule **EV-MATCH**, we simplify an example from **Section 2**.

$$v \mapsto^2 W, w \mapsto^2 W \vdash_v \text{match } (v, w)^2 \text{ as } (x, y) \Rightarrow \text{return}^1 x \Downarrow \text{return}^1 W$$

This match expression matches on 2 of the pair (v, w) , meaning that it takes the environment $v \mapsto^1 W, w \mapsto^1 W$ required to evaluate the pair and multiplies it by 2. The match body then evaluates with environment $x \mapsto^2 W, y \mapsto^2 W$, but in actuality only requires 1 of x and 0 of y . Therefore, evaluating the match body requires rule **EV-CSUB**. Even if the scrutinee weren't duplicated (*i.e.* used $(v, w)^1$ instead of $(v, w)^2$), the evaluation would still be imprecise because w is not used.

For rule **EV-INL** (and likewise rule **EV-INR**), we consider a similar overapproximation.

$$w \mapsto^2 W \vdash_v \text{case } (\text{inl } w)^2 \text{ of } \text{inl } x \Rightarrow \text{return}^1 x \mid \text{inr } y \Rightarrow \text{return}^1 y \Downarrow \text{return}^1 W$$

This case expression matches on 2 of the injection $\text{inl } w$, multiplying the environment $w \mapsto^1 W$ by 2. The first branch is evaluated with environment $x \mapsto^2 W$, but only requires 1 of x . Again, evaluating that branch requires rule **EV-CSUB**.

Imprecision from uses of thunk values is best illustrated together with rule **EV-APP**.

$$w \mapsto^2 W \vdash_v (\lambda x. \text{let } y \leftarrow x! \text{ in let } z \leftarrow x! \text{ in return}^1 z) \{ \text{return}^1 w \}^2 \Downarrow \text{return}^1 W$$

First, the function application evaluates its argument down to the thunk $\{w \mapsto^1 W ; \text{return}^1 w\}$, requiring saving the environment $w \mapsto^1 W$. Because the function demands 2 of its argument, this environment of the function application is multiplied by 2. Then the function body evaluates with a mapping to this thunk argument.

$$x \mapsto^2 \{w \mapsto^1 W ; \text{return}^1 w\} \vdash_v \text{let } y \leftarrow x! \text{ in let } z \leftarrow x! \text{ in return}^1 z \Downarrow \text{return}^1 W$$

Now we evaluate the first let binding, which takes 1 of x and forces its thunk, which in turn returns 1 of W . The let binding maps y to this value.

$$x \mapsto^1 \{w \mapsto^1 W ; \text{return}^1 w\}, y \mapsto^1 W \vdash_v \text{let } z \leftarrow x! \text{ in } \text{return}^1 z \Downarrow \text{return}^1 W$$

The second let binding does the same. Now there are no more uses of x available.

$$x \mapsto^0 \{w \mapsto^1 W ; \text{return}^1 w\}, y \mapsto^1 W, z \mapsto^1 W \vdash_v \text{return}^1 z \Downarrow \text{return}^1 W$$

The body of the inner let binding, however, only uses 1 of z , so there is 1 of y left over in the environment, which must be dropped using rule **EV-CSUB**. This time, the imprecision arises from giving the 2 thunks the same environment even though they use their environments differently.

5 Precise Resource Semantics

To precisely track resources used during evaluation, we introduce a precise resource semantics in Figure 5 which relies on quantity annotations on each constituent closed value in the environment rather than on each variable. In other words, each closed value form W has its own annotation so that environments ρ no longer have an associated resource vector. The return W terminal also does not retain quantity information, since the quantity is stored in the W . The annotations in closed values represent the quantity of constructors available for use; for example, $((\)^2, (\))^3$ is a pair which may be used 3 times, containing a left unit which may be used 2 times and a right unit which may be used 1 time. If the annotated quantity is 0, then a value cannot be used. All rules that eliminate values (**EX-SEQ**, **EX-CASE-INL**, **EX-FORCE**, and **EX-PAIR**) require non-0 annotations.

Quantity annotations prescribe only the top-level structure of the value. For example, it may very well be that a computation demands $\text{inr}^3(\)^0$, meaning that it scrutinizes the value 3 times without using the unit at all. Or, it may be that the value is annotated $\text{inr}^1(\)^4$, meaning the unit is consumed four times, after a single pattern match.

When closed values are created in the semantics (see rules **EX-UNIT**, **EX-INL**, **EX-THUNK**, and **EX-PAIR**), they are given a quantity annotation q that specifies how many times the value can be eliminated during computation (plus some residual if the value appears as part of the result). In a computation of the form $\rho \vdash_p M \Downarrow T$, if all annotations on all subvalues of the result T are 0, then all values created in the computation must be annotated by precisely their number of uses.

The semantics must be prophetic as there is no way to tell, when values are created, what their quantities should be. However, the semantics is deterministic; we show below that there is exactly one quantity that will work for a given residual result.

Rules **EX-VAR** and **EX-UNIT** evaluate as before, but without resource vectors associated to environments, we need some other way of indicating that everything else in the environments cannot be used. We say that unusable closed values are 0-quantity, denoted by $W \approx 0$. There are two equivalent ways of specifying this property:

- We can say that all quantity annotations are 0,⁵ or
- We can define scalar multiplication on a value⁶ $q \cdot W$, and say that W is 0-quantity when there exists some W' such that $W = 0 \cdot W'$ ⁷.

In the proof mechanization, sometimes one definition is easier to work with than the other, but they are interchangeable since they are equivalent. In some theorem statements in the next section, we use scalar multiplication by arbitrary quantities q , not just 0. The multiplication is defined recursively over the value, with $q \cdot \{\rho ; M\}^{q'} = \{\bar{q} \cdot \rho ; M\}^{q \cdot q'}$: multiplying a thunk must also multiply its saved environment, since resuming the thunk q times will require up to q of its resources. (Arbitrary scalar multiplication is never used in the precise semantics itself.)

⁵ [cbpv.exact.zero:allzero_VClos] ⁶ [cbpv.exact.add:mul_VClos] ⁷ [cbpv.exact.zero:iszero_VClos]

$W ::= ()^q \mid \text{inl}^q W \mid \text{inr}^q W \mid (W_1, W_2)^q \mid \{\rho; M\}^q \mid \zeta$			
$T ::= \text{return } W \mid \rho; \lambda x. M \mid \rho; \langle M_1, M_2 \rangle$			
$\rho ::= \emptyset \mid \rho, x \mapsto W$			
$\boxed{\vdash_P V \Downarrow W}$		$\boxed{\rho \vdash_P M \Downarrow T}$	
<i>(precise evaluation)</i>			
EX-VAR	EX-UNIT	EX-INL	EX-THUNK
$\frac{\forall y \neq x, \rho(y) \approx 0}{\rho \vdash_P x \Downarrow \rho(x)}$	$\frac{\forall x, \rho(x) \approx 0}{\rho \vdash_P () \Downarrow ()^q}$	$\frac{\rho \vdash_P V \Downarrow W}{\rho \vdash_P \text{inl } V \Downarrow \text{inl}^q W}$	$\frac{}{\rho \vdash_P \{M\} \Downarrow \{\rho; M\}^q}$
EX-FORCE	EX-RET	EX-ABS	
$\frac{\rho \vdash_P V \Downarrow \{\rho'; M\}^1 \quad \rho' \vdash_P M \Downarrow T}{\rho \vdash_P V! \Downarrow T}$	$\frac{q \neq 0 \quad \rho \vdash_P V \Downarrow W}{\rho \vdash_P \text{return}^q V \Downarrow \text{return } W}$	$\frac{}{\rho \vdash_P \lambda x. M \Downarrow \rho; \lambda x. M}$	
EX-PAIR	EX-LET	EX-APP	
$\frac{\rho_1 + \rho_2 = \rho \quad \rho_1 \vdash_P V_1 \Downarrow W_1 \quad \rho_2 \vdash_P V_2 \Downarrow W_2}{\rho \vdash_P (V_1, V_2) \Downarrow (W_1, W_2)^q}$	$\frac{\rho_1 + \rho_2 = \rho \quad \rho_1 \vdash_P M_1 \Downarrow \text{return } W \quad \rho_2, x \mapsto W \vdash_P M_2 \Downarrow T}{\rho \vdash_P \text{let } x \leftarrow M_1 \text{ in } M_2 \Downarrow T}$	$\frac{q \neq 0 \quad \rho_1 + \rho_2 = \rho \quad \rho_1 \vdash_P M \Downarrow \rho'; \lambda x. M' \quad \rho_2 \vdash_P V \Downarrow W \quad \rho', x \mapsto W \vdash_P M' \Downarrow T}{\rho \vdash_P M V^q \Downarrow T}$	
EX-SEQ	EX-CASE-INL		
$\frac{q \neq 0 \quad \rho_1 + \rho_2 = \rho \quad \rho_1 \vdash_P V \Downarrow ()^q \quad \rho_2 \vdash_P M \Downarrow T}{\rho \vdash_P V^q; M \Downarrow T}$	$\frac{q \neq 0 \quad \rho_1 + \rho_2 = \rho \quad \rho_1 \vdash_P V \Downarrow \text{inl}^q W \quad \rho_2, x_1 \mapsto W \vdash_P M_1 \Downarrow T}{\rho \vdash_P \text{case } V^q \text{ of } \text{inl } x_1 \Rightarrow M_1 \mid \text{inr } x_2 \Rightarrow M_2 \Downarrow T}$		
EX-MATCH	EX-MATCH-JUNK		
$\frac{q \neq 0 \quad \rho_1 + \rho_2 = \rho \quad \rho_1 \vdash_P V \Downarrow (W_1, W_2)^q \quad \rho_2, x_1 \mapsto W_1, x_2 \mapsto W_2 \vdash_P M \Downarrow T}{\rho \vdash_P \text{match } V^q \text{ as } (x_1, x_2) \Rightarrow M \Downarrow T}$	$\frac{q = 0 \quad \rho \vdash_P M \Downarrow \rho'; \lambda x. M' \quad \rho', x \mapsto \zeta \vdash_P M' \Downarrow T}{\rho \vdash_P \text{match } V^q \text{ as } (x_1, x_2) \Rightarrow M \Downarrow T}$		
EX-RET-JUNK	EX-SEQ-JUNK	EX-APP-JUNK	
$\frac{\forall x, \rho(x) \approx 0 \quad q = 0}{\rho \vdash_P \text{return}^q V \Downarrow \text{return } \zeta}$	$\frac{q = 0 \quad \rho \vdash_P M \Downarrow T}{\rho \vdash_P V^q; M \Downarrow T}$	$\frac{q = 0 \quad \rho \vdash_P M \Downarrow \rho'; \lambda x. M' \quad \rho', x \mapsto \zeta \vdash_P M' \Downarrow T}{\rho \vdash_P M V^q \Downarrow T}$	

Fig. 5: Big-step precise resource semantics (excerpt) [cbpv.exact.semantics:EvalVal, EvalComp]

When rule **EX-THUNK** evaluates q thunks by saving its environment ρ , it is saying that the environment contains precisely the total amount of resources required for all q uses of that thunk; each use of the thunk may use different amounts of resources, so we don't count uses via multiplication, which would introduce imprecision. The quantity q of thunks being independent of the quantities inside ρ is the same principle as the quantity of injections being independent of the quantities inside the injected value.

$$\begin{array}{c}
\boxed{W_1 + W_2 = W} \quad \boxed{\rho_1 + \rho_2 = \rho} \quad \text{(adding)} \\
\\
\text{ADD-ENV} \quad \frac{\forall x, \rho_1(x) + \rho_2(x) = \rho(x)}{\rho_1 + \rho_2 = \rho} \quad \text{ADD-UNIT} \quad \frac{()^{q_1} + ()^{q_2} = ()^{q_1+q_2}}{()^{q_1} + ()^{q_2} = ()^{q_1+q_2}} \quad \text{ADD-THUNK} \quad \frac{\rho_1 + \rho_2 = \rho}{\{\rho_1; M\}^{q_1} + \{\rho_2; M\}^{q_2} = \{\rho; M\}^{q_1+q_2}} \\
\\
\text{ADD-PAIR} \quad \frac{W_{11} + W_{12} = W_1 \quad W_{21} + W_{22} = W_2}{(W_{11}, W_{21})^{q_1} + (W_{12}, W_{22})^{q_2} = (W_1, W_2)^{q_1+q_2}} \quad \text{ADD-JUNK} \quad \frac{}{\downarrow + \downarrow = \downarrow} \\
\\
\text{ADD-INL} \quad \frac{W_1 + W_2 = W}{\text{inl}^{q_1} W_1 + \text{inl}^{q_2} W_2 = \text{inl}^{q_1+q_2} W}
\end{array}$$

Fig. 6: Adding closed values and environments (excerpt) [cbpv.exact.add:add_VCloS,add_env]

In rule **EX-PAIR**, we need to somehow assert that the overall resources used to evaluate a pair are the combination of those used to evaluate both components of the pair, but we now have no resource vectors to add. Instead, we define new judgements to describe when an environment or a closed value is the addition of two others, written $\rho_1 + \rho_2 = \rho$ and $W_1 + W_2 = W$, whose rules are given in Figure 6. Intuitively, environments are added pointwise, and closed values may be added when they have the same “shape”, with their corresponding quantities added together.

In rule **ADD-THUNK**, adding two thunks also adds their respective environments. This allows a thunk that uses its resources in different ways to record its total resource usage precisely. Revisiting the last imprecise example from Section 4.1, using a concrete unit value for W , we can now evaluate the following thunk uses precisely:

$$x \mapsto \{w \mapsto ()^1; \text{return}^1 w\}^2 \vdash_P \text{let } y \leftarrow x! \text{ in let } z \leftarrow x! \text{ in return}^1 z \Downarrow \text{return}^1 ()^1$$

The thunk in x is used precisely twice, but the resources used by the thunked computation get split asymmetrically between both uses:

$$\{w \mapsto ()^0; \text{return}^1 w\}^1 + \{w \mapsto ()^1; \text{return}^1 w\}^1 = \{w \mapsto ()^1; \text{return}^1 w\}^2$$

The first forced thunk uses 0 of the returned unit y , while the second forced thunk uses 1 of the returned unit z . The quantities on closed values in environments need not match annotations in the syntax, as is the case for the first thunk: it contains a computation that claims to return 1 of w , but in reality requires 0 of them. Rule **EX-RET** ignores the annotation as it gets dropped altogether.

Annotations on return terminals can be dropped because quantities are instead tracked in the value itself. In rule **EX-LET**, evaluating the body of the let computation requires some returned value W with quantity annotations, which is then demanded from the let-bound computation. There is no need to annotate the binding variable with a quantity. Similarly, function application with rule **EX-APP** also ignores the annotation q on its argument beyond checking $q \neq 0$. The resources required by the function body are demanded from evaluating the function argument with no involvement of q .

Reading the precise computation evaluation rules from premises to conclusion, the resources required of a computation are collected by adding together those required by all subcomputations. Alternatively, reading rules from conclusion to premises, the resources provided to a computation can be disjointly split apart and provided to subcomputations independently. When eliminating q values in rules **EX-SEQ**, **EX-MATCH**, **EX-CASE-INL**, and **EX-CASE-INR**, the premises evaluating the value demand q of those values.

In contrast, by rule **EX-FORCE**, forcing a thunk uses that thunk exactly 1 time, as it runs an entire suspended computation rather than only destructing a value. Otherwise, it would be unclear what it means to run q computations at once, or how q results of computations can be collected.

The junk rules **Ex-MATCH-JUNK**, **Ex-RET-JUNK**, **Ex-SEQ-JUNK**, and **Ex-APP-JUNK** remain dependent on the quantity annotations being 0. This ensures that the precise semantics evaluates the same way as the vector semantics independent of the environment quantities or resource vectors involved, which we show in [Section 6.1](#).

Examples of precise evaluation. Revisiting variations on the remaining examples from [Section 4.1](#), evaluating a match expression that uses pair components asymmetrically needs precisely the resources actually used.

$$v \mapsto ()^1, w \mapsto ()^0 \vdash_P \text{match}(v, w)^2 \text{ as } (x, y) \Rightarrow \text{return}^1 x \Downarrow \text{return}()^1$$

The pair evaluates to $((())^1, ())^2$ according to what is needed by the match body in rule **Ex-MATCH**, which is $x \mapsto ()^1, y \mapsto ()^0$. Similarly, evaluating a case expression that doesn't use the injection requires the precise amount of resources. The injection evaluates to $\text{inl}^2()^0$ as x is not used.

$$w \mapsto ()^0 \vdash_P \text{case}(\text{inl } w)^2 \text{ of } \text{inl } x \Rightarrow \text{return}^1() \mid \text{inr } y \Rightarrow \text{return}^1() \Downarrow \text{return}()^1$$

On typing. A value or computation with incorrect return annotations (and therefore is ill typed) may still evaluate using the precise evaluation relation because the quantities aren't used for anything other than to decide whether to use rule **Ex-RET** or rule **Ex-RET-JUNK**. For instance,

$$x \mapsto ()^2 \vdash_P \text{let } y \leftarrow \text{return}^1 x \text{ in } y; \text{return}^1 y \Downarrow \text{return}()^1$$

is a derivable precise evaluation since y is used 2 times and indeed is provided 2 units by x , but this computation cannot be typed since $\text{return}^1 x$ has the wrong quantity. Because the vector semantics uses quantity annotations in computations to annotate its resource vector, this computation cannot evaluate under the vector semantics.

6 Properties of the Precise Semantics

6.1 Quantity Erasure

The most basic property we desire of the precise semantics is to know that, all instrumentation aside, it evaluates values and computations the same way as the vector semantics. We relate the two via a *raw* resource semantics, denoted by $\rho \vdash_R V \Downarrow W$ and $\rho \vdash_R M \Downarrow T$, which describe how values and computations evaluate under raw environments to raw results with no coefficient tracking. Their rules are the same as those of the vector semantics, only with all resource vectors omitted; alternatively, they are the same as those of the precise semantics, only with all quantities inside of closed values omitted. The complete listing of rules is given in [Figure 13](#) of [Appendix B](#).

Using $|W|$, $|T|$, and $|\rho|$ to denote the erasure of annotated quantities and resource vectors, we show that erasing both vector and precise semantics derivations preserves evaluation.

THEOREM 6.1 (ERASURE OF RESOURCE VECTOR SEMANTICS). [\[cbpv.vector_raw:V.Eval*_erase\]](#)

- If $\gamma \cdot \rho \vdash_V V \Downarrow W$ then $|\rho| \vdash_R V \Downarrow |W|$.
- If $\gamma \cdot \rho \vdash_V M \Downarrow T$ then $|\rho| \vdash_R M \Downarrow |T|$.

THEOREM 6.2 (ERASURE OF PRECISE RESOURCE SEMANTICS). [\[cbpv.exact_raw:Eval*_erase\]](#)

- If $\rho \vdash_P V \Downarrow W$ then $|\rho| \vdash_R V \Downarrow |W|$.
- If $\rho \vdash_P M \Downarrow T$ then $|\rho| \vdash_R M \Downarrow |T|$.

Then by determinism of the raw semantics, the vector semantics and the precise semantics produce the same raw results.

LEMMA 6.3 (DETERMINISM OF RAW SEMANTICS). [\[cbpv.raw.semantics:Eval*_det\]](#)

- If $\rho \vdash_R V \Downarrow W_1$ and $\rho \vdash_R V \Downarrow W_2$ then $W_1 = W_2$.

- If $\rho \vdash_R M \Downarrow T_1$ and $\rho \vdash_R M \Downarrow T_2$ then $T_1 = T_2$.

COROLLARY 6.4. `[cbpv.vector_raw:Eval*_erase]`

- Suppose $\gamma \cdot \rho_1 \vdash_V V \Downarrow W_1$ and $\rho_2 \vdash_P V \Downarrow W_2$. If $|\rho_1| = |\rho_2|$ then $|W_1| = |W_2|$.
- Suppose $\gamma \cdot \rho_1 \vdash_V M \Downarrow T_1$ and $\rho_2 \vdash_P M \Downarrow T_2$. If $|\rho_1| = |\rho_2|$ then $|T_1| = |T_2|$.

In the other direction, given a raw evaluation and a desired precise annotation of its result, it is possible to find an annotation of the environment to produce a precise evaluation: the annotated result constrains the resources demanded. This is proven by induction on the derivation of raw evaluation, working backwards, from the leaves of the computation to the root. That way, the quantities decorating values can be determined from their uses and added together in the environments to produce the necessary annotation when the value is created.

LEMMA 6.5 (PRECISE ANNOTATION OF RAW SEMANTICS). `[cbpv.exact_raw:Eval*_annotate]`

- Suppose $\rho \vdash_R V \Downarrow W$. Then for any W' such that $|W'| = W$, there exists some ρ' such that $\rho' \vdash_P V \Downarrow W'$, where $|\rho'| = \rho$.
- Suppose $\rho \vdash_R M \Downarrow T$. Then for any T' such that $|T'| = T$, there exists some ρ' such that $\rho' \vdash_P M \Downarrow T'$, where $|\rho'| = \rho$.

On the other hand, providing the environment annotations to find an annotation of the result is not always possible, since the environment annotations could be wrong. For example, $x \mapsto () \vdash_R x \Downarrow ()$ is a valid raw evaluation, but using $x \mapsto^0 ()$ cannot yield a valid precise evaluation.

In contrast to precise evaluation, it is unclear how to produce a vector evaluation from a raw evaluation for reasons we discuss in [Appendix C](#).

6.2 Unique Environments

When we say that the precise environments are unique, we mean that given any evaluation from a fixed value or computation to a result, there is only one possible set of resources for a particular environment. That is, if $x \mapsto \{\rho; M'\}^q \vdash_P M \Downarrow T$, for instance, then there is only one possible quantity q . We formally state this in terms of *similarity*, which ignores quantities via multiplication by 0. Multiplication is extended from closed values and environments to terminals $q \cdot T$ by $q \cdot \text{return } W = \text{return } (q \cdot W)$ and $q \cdot \rho; M = \bar{q} \cdot \rho; M$ ⁸.

Definition 6.6 (Similar results and environments). `[cbpv.exact.similar:same_{env,*Clos}]`

- Two environments are similar, written $\rho_1 \approx \rho_2$, if $\bar{0} \cdot \rho_1 = \bar{0} \cdot \rho_2$.
- Two closed values are similar, written $W_1 \approx W_2$, if $0 \cdot W_1 = 0 \cdot W_2$.
- Two terminals are similar, written $T_1 \approx T_2$, if $0 \cdot T_1 = 0 \cdot T_2$.

Similarity is a precondition to adding closed values and environments: if they can be added, then they must be similar, and if they are similar, then they can be added.

LEMMA 6.7 (SIMILARITY OF ADDABLES). `[cbpv.exact.similar:add_{VClos,env}_same{1,2}]`

If $W_1 + W_2 = W$ then $W_1 \approx W_2 \approx W$, and if $\rho_1 + \rho_2 = \rho$ then $\rho_1 \approx \rho_2 \approx \rho$.

LEMMA 6.8 (ADDABILITY OF SIMILARS). `[cbpv.exact.similar:add_{VClos,env}_exists]`

If $W_1 \approx W_2$ then $W_1 + W_2 = W$ for some W , and if $\rho_1 \approx \rho_2$ then $\rho_1 + \rho_2 = \rho$ for some ρ .

We need lemmas stating that evaluation of the same value or computation under similar environments produce similar results, which captures the intuition that using values of the same “shape” in the environment produces the same results up to differing quantities.

⁸ `[cbpv.exact.add:mul_env,mul_CClos]`

LEMMA 6.9 (PRECISE EVALUATION PRESERVES SIMILARITY). [cbpv.exact.semantics:Eval*_same]

- Suppose $\rho_1 \vdash_P V \Downarrow W_1$ and $\rho_2 \vdash_P V \Downarrow W_2$. If $\rho_1 \approx \rho_2$ then $W_1 \approx W_2$.
- Suppose $\rho_1 \vdash_P M \Downarrow T_1$ and $\rho_2 \vdash_P M \Downarrow T_2$. If $\rho_1 \approx \rho_2$ then $T_1 \approx T_2$.

Now we can show that if two evaluations use similar environments, then their environments are in fact equal. This means that there is no approximation of resources during precise evaluation, unlike in the vector semantics.

THEOREM 6.10 (ENV. DETERMINISM OF PRECISE SEMANTICS). [cbpv.exact.semantics:Eval*_uniq]

- Suppose $\rho_1 \vdash_P V \Downarrow W$ and $\rho_2 \vdash_P V \Downarrow W$. If $\rho_1 \approx \rho_2$ then $\rho_1 = \rho_2$.
- Suppose $\rho_1 \vdash_P M \Downarrow T$ and $\rho_2 \vdash_P M \Downarrow T$. If $\rho_1 \approx \rho_2$ then $\rho_1 = \rho_2$.

While by [Theorem 6.10](#) the result of evaluation determines how many resources are required, and by [Lemma 6.3](#) what the resources are determines what the result is, we do *not* have that the amount of resources provided determines the quantities in the result. This is because the quantities in the environment only constrain how the values in the environment are used, and not any other value literals used during evaluation. For instance, $\emptyset \vdash_P () \Downarrow ()^q$ is valid for any q , and $\emptyset \vdash_P \text{return}^\omega() \Downarrow \text{return}()^q$ is valid for any q as well. Consequently, precise evaluation is not deterministic in the quantities of its result. This is alright because its purpose is to track the precise resources used during evaluation, which is deterministic when the result is fixed.

6.3 Type Safety (Vector Semantics)

Prior to type safety of the precise semantics, we first address that of the vector semantics. As our type system and vector semantics haven't changed substantially from prior work by [Torczon et al. \[2024\]](#), the revisions to the logical relation and to semantic typing are minor. Therefore, we only briefly summarize the results here to establish notation.

Definition 6.11 (Semantic typing with respect to resource vector semantics). A term is semantically well typed at a type under a resource vector and a typing context if, for all semantically well-formed environments with respect to the vector and context, the term evaluates under the environment to a result that is in the logical interpretation of the type. The relevant notations are defined below, with the logical relations $\mathcal{W}[A]$ and $\mathcal{T}[B]$ defined in [Figure 14 of Appendix D](#).

$$\gamma \cdot \rho \models V \in \mathcal{V}[A] \Leftrightarrow \exists W, (\gamma \cdot \rho \vdash_V V \Downarrow W) \wedge (W \in \mathcal{W}[A]) \quad (\text{logical value interpretation})$$

$$\gamma \cdot \rho \models M \in \mathcal{M}[B] \Leftrightarrow \exists T, (\gamma \cdot \rho \vdash_V M \Downarrow T) \wedge (T \in \mathcal{T}[B]) \quad (\text{logical computation interpretation})$$

$$\gamma \cdot \rho \models \Gamma \Leftrightarrow \forall x, (\gamma(x) = 0) \vee (\rho(x) \in \mathcal{W}[\Gamma(x)]) \quad (\text{semantic environment formation})$$

$$\gamma \cdot \Gamma \models V : A \Leftrightarrow \forall \rho, (\gamma \cdot \rho \models \Gamma) \Rightarrow (\gamma \cdot \rho \models V \in \mathcal{V}[A]) \quad (\text{semantic value typing})$$

$$\gamma \cdot \Gamma \models M : B \Leftrightarrow \forall \rho, (\gamma \cdot \rho \models \Gamma) \Rightarrow (\gamma \cdot \rho \models M \in \mathcal{M}[B]) \quad (\text{semantic computation typing})$$

[cbpv.vector.semtyping:LRW,LRM,SemEnv,SemVWt,SemCWt]

We highlight four properties of semantic environment formation that motivate the axioms we impose upon our preordered semiring of coefficients.

LEMMA 6.12 (PROPERTIES OF SEMANTIC ENVIRONMENT FORMATION).

- (1) If $(\gamma_1 + \gamma_2) \cdot \rho \models \Gamma$ then $\gamma_1 \cdot \rho \models \Gamma$ and $\gamma_2 \cdot \rho \models \Gamma$. [cbpv.vector.semtyping:SemEnv_add]
- (2) If $q \cdot \gamma \cdot \rho \models \Gamma$ then $q = 0$ or $\gamma \cdot \rho \models \Gamma$. [cbpv.vector.semtyping:SemEnv_mul]
- (3) If $\gamma \cdot \rho \models \Gamma$ and $\gamma \leq \gamma'$ then $\gamma' \cdot \rho \models \Gamma$. [cbpv.vector.semtyping:SemEnv_le]
- (4) If $\gamma \cdot \rho \models \Gamma$ and $\gamma(x) = 1$ then $\rho(x) \in \mathcal{W}[\Gamma(x)]$. [cbpv.vector.semtyping:SemEnv_Qone]

$$\begin{array}{c}
\boxed{W_1 \leq W_2} \quad \boxed{T_1 \leq T_2} \quad \boxed{\rho_1 \leq \rho_2} \quad (subsumption) \\
\\
\text{SUB-UNIT} \quad \text{SUB-INL} \quad \text{SUB-PAIR} \quad \text{SUB-THUNK} \\
\frac{q_1 \leq q_2}{()^{q_1} \leq ()^{q_2}} \quad \frac{q_1 \leq q_2 \quad W_1 \leq W_2}{\text{inl}^{q_1} W_1 \leq \text{inl}^{q_2} W_2} \quad \frac{q_1 \leq q_2 \quad W_{11} \leq W_{12} \quad W_{21} \leq W_{22}}{(W_{11}, W_{21})^{q_1} \leq (W_{12}, W_{22})^{q_2}} \quad \frac{q_1 \leq q_2 \quad \rho_1 \leq \rho_2}{\{\rho_1; M\}^{q_1} \leq \{\rho_2; M\}^{q_2}} \\
\\
\text{SUB-JUNK} \quad \text{SUB-RET} \quad \text{SUB-CLOS} \quad \text{SUB-ENV} \\
\frac{}{\downarrow \leq \downarrow} \quad \frac{W_1 \leq W_2}{\text{return } W_1 \leq \text{return } W_2} \quad \frac{\rho_1 \leq \rho_2}{\rho_1; M \leq \rho_2; M} \quad \frac{\forall x, \rho_1(x) \leq \rho_2(x)}{\rho_1 \leq \rho_2}
\end{array}$$

Fig. 7: Subsumption of results and environments (excerpt) [cbpv.exact.sub:sub_env, sub_*Clos]

Proof (sketch). By cases on the hypotheses. WLOG, pick an arbitrary x .

- (1) In the case where $(\gamma_1 + \gamma_2)(x) = 0$, deduce $\gamma_1(x) = 0$ and $\gamma_2(x) = 0$ by zerosumfreeness.
- (2) In the case where $q \neq 0$ and $q \cdot \gamma(x) = 0$, deduce $\gamma(x) = 0$ by entirety.
- (3) In the case where $\gamma(x) = 0$, deduce $\gamma'(x) = 0$ by nonnegativity.
- (4) Since $\gamma(x) = 1$, deduce $\gamma(x) \neq 0$ by nontriviality. ■

The fundamental theorem of soundness of syntactic typing with respect to semantic typing is proven as usual by mutual induction on the value and computation typing derivations.

THEOREM 6.13 (FUNDAMENTAL TH'M OF SEMANTIC TYPING). [cbpv.vector.semtyping:fundamental]

- If $\gamma \cdot \Gamma \vdash V : A$ then $\gamma \cdot \Gamma \models V : A$.
- If $\gamma \cdot \Gamma \vdash M : B$ then $\gamma \cdot \Gamma \models M : B$.

Type safety with respect to the vector semantics states that well typed values and computations always evaluate to results using the vector semantics, and follows directly from [Theorem 6.13](#).

COROLLARY 6.14 (TYPE SAFETY (RESOURCE VECTOR SEMANTICS)). [cbpv.vector.semtyping:safety]

- If $\gamma \cdot \Gamma \vdash V : A$, then for any ρ where $\gamma \cdot \rho \models \Gamma$, there exists some W such that $\gamma \cdot \rho \vdash_v V \Downarrow W$.
- If $\gamma \cdot \Gamma \vdash M : B$, then for any ρ where $\gamma \cdot \rho \models \Gamma$, there exists some T such that $\gamma \cdot \rho \vdash_v M \Downarrow T$.

6.4 Type Safety (Precise Semantics)

Having designed a new precise semantics for the type system, we need to show that type safety still holds with respect to this semantics. Rather than proving another fundamental theorem with another logical relation, we instead show that if a value or a computation evaluates to a result by the vector semantics, then it evaluates to a result by the precise semantics, and combine this fact with the original type safety theorem.

Formally stating the former theorem requires two auxiliary definitions. First, since the vector semantics has subsumption rules while the precise semantics does not, we need a notion of subsumption on precise environments $\rho_1 \leq \rho_2$, closed values $W_1 \leq W_2$, and terminals $T_1 \leq T_2$. Their derivation rules resemble those of addition, and are given in [Figure 7](#).

Subsumption is a reflexive, transitive relation⁹ that respects multiplication and similarity and satisfies an inversion property on environments.

LEMMA 6.15 (PROPERTIES OF SUBSUMPTION).

- Suppose $\gamma_1 \leq \gamma_2$ and $q_1 \leq q_2$. If $\rho_1 \leq \rho_2$, $W_1 \leq W_2$, and $T_1 \leq T_2$, then $\gamma_1 \cdot \rho_1 \leq \gamma_2 \cdot \rho_2$, $q_1 \cdot W_1 \leq q_2 \cdot W_2$, and $q_1 \cdot T_1 \leq q_2 \cdot T_2$, respectively.
- If $\rho_1 \leq \rho_2$, $W_1 \leq W_2$, and $T_1 \leq T_2$, then $\rho_1 \approx \rho_2$, $W_1 \approx W_2$, and $T_1 \approx T_2$, respectively.

⁹ [cbpv.exact.sub:sub_{env,*Clos}_{refl,trans}]

- If $\rho_1, x_1 \mapsto W_1 \leq \rho$, then $\rho = \rho_2, x_2 \mapsto W_2$ such that $\rho_1 \leq \rho_2$ and $W_1 \leq W_2$.

[cbpv.exact.sub:Qle_mul_sub_{env,*Clos}, sub_{env,*Clos}_{same}, sub_env_consL_inv]

Subsumption also respects addition, but we prove a particular version with quantity addition on the left and precise closed value and environment addition on the right, which are the particular forms needed for theorems later on.

LEMMA 6.16 (SUBSUMPTION RESPECTS ADDITION).

- Let $W_1 + W_2 = W_3$. If $q_1 \cdot W \leq W_1$ and $q_2 \cdot W \leq W_2$, then $(q_1 + q_2) \cdot W \leq W_3$.
- Let $\rho_1 + \rho_2 = \rho_3$. If $\gamma_1 \cdot \rho \leq \rho_1$ and $\gamma_2 \cdot \rho \leq \rho_2$, then $(\gamma_1 + \gamma_2) \cdot \rho \leq \rho_3$.

[cbpv.exact.sub:sub_mul_add] [cbpv.exact.inject:sub_inject_add_env]

The second auxiliary definition we need embeds resource results and environments into precise results and environments, respectively, to strengthen the induction hypotheses we need, as stating that vector and precise semantics evaluate to unrelated results isn't strong enough. The embeddings $\vec{\rho}$, $\vec{W}$, and $\vec{T}$ from vector to precise denote inserting the quantity 1 everywhere.¹⁰ This aligns with the interpretation of a 1 vector pair, for instance, as allowing 1 use for matching on the pair and 1 use each of its components.

We can now assert that we can produce a precise value evaluation from a vector value evaluation. This result is needed for the analogous property for computation evaluation (Theorem 6.18).

THEOREM 6.17 (VECTOR TO PRECISE VALUE EVAL.). [cbpv.exact_vector:EvalVal_vector]

Suppose $\gamma \cdot \rho \vdash_v V \Downarrow W$. Then for any q and W' such that $q \cdot \vec{W} \leq W'$, there exists some ρ' such that $\rho' \vdash_p V \Downarrow W'$, where $q \cdot \gamma \cdot \vec{\rho} \leq \rho'$.

Because the vector semantics is imprecise, we cannot expect merely embedding the environment ρ to be precise enough for the precise semantics, so we output a subsuming precise environment $\gamma \cdot \vec{\rho} \leq \rho'$, which is the unique and therefore most precise environment, using equal or fewer resources than the embedding. (We already proved uniqueness in Theorem 6.10, so it isn't stated here.) Theorem 6.18 later instantiates this theorem with a closed value that comes from subsumption inversion, so we require a subsuming precise closed value $\vec{W} \leq W'$. Finally, computation rules also multiply closed values by arbitrary quantities q before extending the environment, so we further generalize by multiplying the embeddings by q .

Next, the theorem producing a precise computation evaluation from a vector computation evaluation is stated similarly, except we don't generalize by a multiplication by q , as terminals are never multiplied during evaluation.

THEOREM 6.18 (VECTOR TO PRECISE COMPUTATION EVAL.). [cbpv.exact_vector:EvalComp_vector]

Suppose $\gamma \cdot \rho \vdash_v M \Downarrow T$. Then for any T' such that $\vec{T} \leq T'$, there exists some ρ' such that $\rho' \vdash_p M \Downarrow T'$, where $\gamma \cdot \vec{\rho} \leq \rho'$.

Combining Theorems 6.17 and 6.18 with Corollary 6.14 and Theorem 6.10, we get type safety of precise evaluation: if a term is well typed, then using any semantically well-formed environment with respect to the typing context, the term evaluates to a result under a uniquely subsuming precise environment (i.e. a unique ρ' among all that satisfy $\gamma \cdot \vec{\rho} \leq \rho'$).

¹⁰ [cbpv.exact.inject:inject_env, inject_VClos, inject_CClos]

$\boxed{W_1 \sqsubseteq W_2}$	$\boxed{T_1 \sqsubseteq T_2}$	$\boxed{\rho_1 \sqsubseteq \rho_2}$	(discarding in environments and results)	
D-DISCARD $\frac{Q(W) = 0}{W \sqsubseteq \zeta}$	D-JUNK $\frac{}{\zeta \sqsubseteq \zeta}$	D-UNIT $\frac{}{()^q \sqsubseteq ()^q}$	D-INL $\frac{W_1 \sqsubseteq W_2}{\text{inl}^q W_1 \sqsubseteq \text{inl}^q W_2}$	D-PAIR $\frac{W_{11} \sqsubseteq W_{12} \quad W_{21} \sqsubseteq W_{22}}{(W_{11}, W_{21})^q \sqsubseteq (W_{12}, W_{22})^q}$
D-THUNK $\frac{\rho_1 \sqsubseteq \rho_2}{\{\rho_1; M\}^q \sqsubseteq \{\rho_2; M\}^q}$	D-RET $\frac{W_1 \sqsubseteq W_2}{\text{return } W_1 \sqsubseteq \text{return } W_2}$	D-CLOS $\frac{\rho_1 \sqsubseteq \rho_2}{\rho_1; M \sqsubseteq \rho_2; M}$	D-ENV $\frac{\forall x, \rho_1(x) \sqsubseteq \rho_2(x)}{\rho_1 \sqsubseteq \rho_2}$	

Fig. 8: Discarding inaccessible closed values (excerpt) [cbpv.exact.discard:disc_env,disc_*Clos]

THEOREM 6.19 (TYPE SAFETY (PRECISE RESOURCE SEMANTICS)). [cbpv.exact_vector:Eval*_safety]

- If $\gamma \cdot \Gamma \vdash V : A$, then for any ρ where $\gamma \cdot \rho \models \Gamma$, there exist W and a unique ρ' such that $\gamma \cdot \rho \leq \rho'$ and $\rho' \vdash_P V \Downarrow W$.
- If $\gamma \cdot \Gamma \vdash M : B$, then for any ρ where $\gamma \cdot \rho \models \Gamma$, there exist T and a unique ρ' such that $\gamma \cdot \rho \leq \rho'$ and $\rho' \vdash_P M \Downarrow T$.

6.5 Discarding

As a corollary of type safety, we know that variables with quantities $q = 0$ will never be used, so evaluations can proceed with those variables set to junk values.

COROLLARY 6.20 (INACCESSIBLE JUNK VARIABLES). [cbpv.vector.semtyping:inaccessible]
If $x : {}^0A \vdash M : B$ then $x \mapsto \zeta \vdash_P M \Downarrow T$ for some terminal T .

However, evaluation is also possible under environments with closed values whose top-level quantities might not satisfy $q = 0$, but whose subvalues' quantities might:

$$x \mapsto \text{inl}^1({})^0 \vdash_P \text{case } x^1 \text{ of } \text{inl } y_1 \Rightarrow \text{return}^1({}) \mid \text{inr } y_2 \Rightarrow \text{return}^1 y_2 \Downarrow \text{return}({})^1.$$

It ought to be possible to discard any inaccessible-quantity subvalue, not just the top-level environment values. Doing so can be viewed as deallocation, since the space taken up by the unusable value gets reclaimed, and the reference is marked as freed using the junk value as a placeholder.

We define a new reflexive, transitive relation on environments and results in Figure 8 describing how inaccessible closed values get discarded and replaced by junk values. $\rho_1 \sqsubseteq \rho_2$ says that ρ_2 may have discarded any amount of junk values found in ρ_1 , and similarly for results. We use $Q(W)$ to denote the outermost quantity of a closed value, with $Q(\zeta) = 0$.

This relation is unrelated to subsumption. From Lemma 6.15, subsuming environments are similar, and therefore have more restrictive quantities while keeping the shape of values intact; discarding environments, in contrast, touch no quantities, but may turn values into junk, and are therefore *not* similar. The key rule is D-DISCARD, which discards inaccessible closed values W where $Q(W) = 0$. The remaining rules form its compatible closure, which justifies discarding $\text{inl}^1({})^0 \sqsubseteq \text{inl}^1 \zeta$ as in the earlier example. Values in thunk environments may be also discarded by rule D-THUNK:

$$\{x \mapsto ({}^0; (\lambda y. \text{return}^1({})) x^0)^1\} \sqsubseteq \{x \mapsto \zeta; (\lambda y. \text{return}^1({})) x^0\}^1$$

While discarding doesn't preserve similarity, it does preserve 0ness along the direction of discarding.

LEMMA 6.21 (DISCARDING PRESERVES 0). [cbpv.exact.discard:disc_iszero_VClos]
Suppose $W_1 \sqsubseteq W_2$. If $W_1 \approx 0$ then $W_2 \approx 0$.

If a closed value consists of an addition, then discarding in that value consists of discarding from each component of the addition, and similarly for environments: discarding preserves the way that quantities are split across an addition. To compute the concrete closed values that exist, these are

proven first by induction on W , then by inversion on the derivations of addition and discarding. They also require zerosumfreeness to split apart added 0-quantity closed values.

LEMMA 6.22 (DISCARDING PRESERVES ADDITION). $[\text{cbpv.exact.discard:disc_add}\{\text{VClos}, \text{env}\}]$

- Suppose $W_1 + W_2 = W$. If $W \sqsubseteq W'$, then there exist W'_1 and W'_2 such that $W'_1 + W'_2 = W'$, $W_1 \sqsubseteq W'_1$, and $W_2 \sqsubseteq W'_2$.
- Suppose $\rho_1 + \rho_2 = \rho$. If $\rho \sqsubseteq \rho'$, then there exist ρ'_1 and ρ'_2 such that $\rho'_1 + \rho'_2 = \rho'$, $\rho_1 \sqsubseteq \rho'_1$, and $\rho_2 \sqsubseteq \rho'_2$.

These properties allow us to show that discarding is safe. A term evaluates even if we remove 0-quantity values from the environment. Furthermore, while the inaccessible values may be discarded from the new result, all nonzero subvalues will be preserved. For example, the x in $x \mapsto ()^0, y \mapsto ()^1 \vdash_P \text{return}^1(x, y) \Downarrow \text{return}((()^0, ()^1)^1$ can be discarded to yield $x \mapsto \frac{1}{2}, y \mapsto ()^1 \vdash_P \text{return}^1(x, y) \Downarrow \text{return}(\frac{1}{2}, ()^1)^1$. Proving this theorem requires nonnegativity and nontriviality to take care of impossible cases when eliminating injections.

THEOREM 6.23 (DISCARDING IN EVALUATIONS). $[\text{cbpv.exact.semantics:Eval*_disc}]$ Let $\rho \sqsubseteq \rho'$.

- If $\rho \vdash_P V \Downarrow W$, then there exists some W' such that $\rho' \vdash_P V \Downarrow W'$ and $W \sqsubseteq W'$.
- If $\rho \vdash_P M \Downarrow T$, then there exists some T' such that $\rho' \vdash_P M \Downarrow T'$ and $T \sqsubseteq T'$.

7 Related Work

Our work directly extends that by [Torczon et al. \[2024\]](#), who add both effect and coeffect tracking to call-by-push-value [\[Levy 2003\]](#), and provide an instrumented semantics for evaluating such programs. We focus on solving the problem of imprecise coeffect instrumentation, and so we omit effect tracking from our type system and from our operational semantics.

A notable difference from the typing rules presented by [Torczon et al.](#) is that their let expressions have a quantity q' associated with the bound variable in addition to the quantity q from the bound expression's type $F^q A$. Their rule **T-LET'** then states that if there are adequate resources to compute $F^q A$ a total of q' times, then the returned result may be used $q \cdot q'$ times.

$$\frac{\gamma_1 \cdot \Gamma \vdash M_1 : F^q A \quad \gamma_2 \cdot \Gamma, x : {}^{q \cdot q'} A \vdash M_2 : B}{(q \cdot \gamma_1 + \gamma_2) \cdot \Gamma \vdash \text{let } x \leftarrow {}^{q'} M_1 \text{ in } M_2 : B} \text{T-LET'}$$

However, this conflates the quantity of values we can compute with the number of times we may use a particular value, which leads to unintuitive behaviour. Taking an example by [Lorenzen et al. \[2024, Sec. 5.1\]](#), consider a computation $\text{open} : \mathbb{T}^1 \rightarrow F^1 \mathbf{File}$ which returns a file handle, and a corresponding computation $\text{close} : \mathbf{File}^1 \rightarrow F^1 \mathbb{T}$ which closes the file. While we have the resources to open as many files as we wish, Rule **T-LET'** would allow closing the same file twice.

$$\emptyset \vdash \text{let } f \leftarrow {}^2 \text{open}() \text{ in let } x_1 \leftarrow \text{close } f \text{ in let } x_2 \leftarrow \text{close } f \text{ in return}^1() : F^1 \mathbb{T}$$

Although our presentation of CBPV with coeffects doesn't deal with anything as complex as file operations, we nonetheless omit quantity scaling from our let expressions to ensure that the language can be extended later without inducing such unintuitive behaviour.

7.1 Type-and-Coeffect Systems

The notion of coeffects as a single (or *flat*) annotation on a typing context was introduced by [Petrick et al. \[2013\]](#), who use them for liveness and dataflow analysis. They go on to generalize coeffects from flat coeffects to per-variable (or *structural*) coeffects [\[Petrick et al. 2014\]](#) for applications such as bounded linearity. Independently, [Ghica and Smith \[2014\]](#) develop a type system for bounded linear logic with a per-variable notion of resource modelled by a semiring, while [Brunel et al. \[2014\]](#) develop a type system for a linear variant of PCF with a modality annotated (or *graded*) by semiring

resources. Gaboardi et al. [2016] then combine a graded monadic modality for effects with a graded comonadic modality for coeffects and explore their interaction in a simply typed setting. The Granule project [Orchard et al. 2019] builds upon their work and implements a type-polymorphic language with graded modalities, GADTs, and bidirectional type checking. While Granule focuses on the implementation, with several concrete modalities, Abel and Bernardy [2020] describe a smaller language Λ^P parametrized over more general graded modalities and provide a relational model, although without considering effects. A relational theory of effects and coeffects combined, but without polymorphism, is developed by Dal Lago and Gavazzo [2022] in their fine-grained call-by-value language $\Lambda_{\Sigma, S}$. There are also a number of works that combine some form of coeffects with dependent types, such as Quantitative Type Theory (QTT) [Atkey 2018; McBride 2016], Graded Modal Dependent Type Theory (GRTT) [Abel et al. 2023; Moon et al. 2021], GRAD [Choudhury et al. 2021], and Dependent Linear Type Theory (DLTT) [Doré 2026].

Algebraic structure of coeffects. Most prior work on coeffects requires them to form either a preordered [Dal Lago and Gavazzo 2022; Gaboardi et al. 2016; Orchard et al. 2019; Torczon et al. 2024] or a partially ordered [Bianchini et al. 2023; Choudhury et al. 2021; Liepelt et al. 2026; Petricek et al. 2014] semiring; like our direct ancestor, we opt for a preordered semiring. Some systems add further axioms on top of this structure for specific properties or applications. In this paper, we consider a *nonnegative* preorder, i.e. $0 \leq q \Rightarrow q = 0$, which is a weaker condition than a *positive* partial order where $q \leq 0$ for all q [Golan 2003]; Bianchini et al. [2026] call this property *discardability*. Like us, the semiring in QTT needs to be *zerosumfree* and *entire*, i.e. $q_1 + q_2 = 0 \Rightarrow q_1 = 0 \wedge q_2 = 0$ and $q_1 \cdot q_2 = 0 \Rightarrow q_1 = 0 \vee q_2 = 0$, although they need them for their substitution lemmas to hold. In Λ^P , coeffects instead form a *meet-semilattice*, which adds a *meet* (i.e. infimum) operation on quantities so that two branches of a case expression using different quantities of a variable can always be unified. Choudhury et al. note that for irrelevance, *zero-unusability* $0 \not\leq q + 1$ suffices, while for linearity, *one-linearity* $1 \not\leq q + 1$ where $q \neq 0$ suffices, and counting for garbage collection requires the semiring to be zerosumfree and entire.

To use coeffects for information flow control and to prove a noninterference theorem, Liepelt et al. [2026] define a *noninterfering ring*, which is a partially ordered ring bounded by $1 \leq q \leq 0$ with *lax idempotence*, i.e. $q \cdot q \leq q$. Because our system only permits using 1-quantity variables and only discards 0-quantity variables, the quantity constraints in premises would need to be revisited to accommodate applications in information flow.

Resource sufficiency. Different instrumented coeffect semantics place different quantity requirements on when there are sufficient resources to be used. Like our predecessor by Torczon et al., our operational semantics is conditional on $q \neq 0$ when eliminating values. Similarly, the fine-grained call-by-value language by Bianchini et al. enforces $q \neq 0$ not just in the operational semantics, but in the type system as well, and consequently has no notion of eliminating a 0-quantity value. In contrast, the semantics of GRAD and Λ^P are both call-by-name, where the heap stores unevaluated expressions, not values, and pattern-matching on a pair, for instance, is not considered a usage if the quantity is 0 and the pair components are not evaluated. In our system, our predecessor's, GRAD, Λ^P , and the system by Liepelt et al., branching expressions such as case and if require quantity $q \leq 1$ on the scrutinee for well-typedness.

7.2 Usage-Instrumented Semantics

Coeffect-aware semantics for discardable resources. Prior work on coeffects provides instrumented/resource-aware semantics that track resources during execution in various ways. However, all of these semantics, like the big-step environment-based CBPV resource semantics of Torczon et al., are imprecise for resource tracking.

Λ^P has a call-by-name heap-based abstract machine, where heap entries are annotated by quantities, and the machine state has an additional quantity tracking permitted usages of the current term being evaluated. GRAD uses a call-by-name small-step heap semantics, where similarly the heap tracks resources and reduction steps have a tracked quantity. Following GRAD, Eriksson et al. [2026] similarly give a call-by-name heap-based abstract machine, this time for a graded modal dependent type theory with recursion. Their results were mechanized using Agda.

Bianchini et al. [2023] give a (fine-grained) call-by-value big-step heap semantics with annotations. While their semantics, like ours, uses closed values whose subvalues have their own quantity annotations, their heaps still annotate each entry with its own quantity, and these quantities get multiplied together. For instance, in their system, the heap $x \mapsto^2 \text{inl}^2 W$ requires $2 \cdot 2$ of W , which is imprecise because each use of x might in reality require a different amount of W .

Precision for nondiscardable resources. If the coefficient order is discrete ($q_1 \leq q_2$ iff $q_1 = q_2$), then the resource vector semantics is already precise. In contrast, in the resource-aware semantics of Bianchini et al., there is no guarantee that resources from a discrete order, which they call *nondiscardable* (i.e. quantities q such that $q \not\leq 0$), are entirely used up. This is because their resource-aware semantics uses a big-step heap semantics, which begins with some program and a resource-annotated heap, and big-steps to a resulting value along with some residual heap, with no requirement that the residual resources be 0.

In followup work [Bianchini et al. 2026], they design a *no-waste* semantics that is precise for nondiscardable resources. Their approach constrains the residual heap according to what is used by the resulting value, enforcing that nondiscardable resources “cannot be wasted”. However, discardable resources may still be wasted, since the constraints are defined in terms of the order on resources. In the language of this paper, their no-waste semantics is precise for discrete orders but not for general orders, as the constraints allow subsumption by the order.

An early precise account of usage for the nondiscardable case appears in the work of Mazurak [2013] on linearity in the type system F° . To characterize what linearity guarantees at run time, Mazurak instruments a call-by-value semantics with *traces* that record when values are *conscribed* (substituted for a variable) and *discharged* (consumed as a value). The resulting run-time linearity theorem shows that values substituted for linear-typed variables are neither duplicated nor discarded over the course of evaluation.

Static cardinality analyses. Cardinality analyses for lazy functional languages count how many times a binding is used, to justify optimizations such as thunk-update avoidance and let-floating: the usage analysis of Turner et al. [1995] and the higher-order cardinality analysis of Sergey et al. [2014]. These analyses model usage with $0/1/\omega$ quantities, rather than an arbitrary semiring, and show that optimizations are sound with respect to a call-by-need model of the heap.

Reference counting. While reference counting may similarly track variables in a heap-based semantics, it tracks the *reachability* of a variable rather than a *usage*, and so is not the same as coefficient tracking. If a computation contains a free variable x , then it is considered to be reachable. Soundness of reference counting guarantees that unreachable variables may be deallocated. However, there may be reachable variables that are never used, such as in the computation $(\lambda y. \text{return } ()) x^0$. Here, static coefficient tracking indicates that x is used 0 times, so its value may be discarded even though it remains reachable. Because unreachable variables may never be used but unused variables may still be reachable, coefficient tracking could safely deallocate more than reference counting.

8 Statically Tracking Coeffects More Precisely

With a precise coeffect-aware operational semantics in place, we can use it to identify places where the type system currently approximates coeffects to see how it could be made more precise.

One possibility is to follow [Bianchini et al. \[2023\]](#) and annotate the subcomponents of value types with quantities, such as $A_1^{q_1} \times A_2^{q_2}$ and $A_1^{q_1} + A_2^{q_2}$, so that the usage of pair and injection components may be statically specified more precisely. Alternatively, we could extend the system with a single graded modal value type $\Box^q A$ that behaves like a unary tuple or sum type carrying q of some value of type A [[Lorenzen et al. 2026](#)]. Then instead of annotating subcomponents, more precise pair and sum types could be implemented as $\Box^{q_1} A_1 \times \Box^{q_2} A_2$ and $\Box^{q_1} A_1 + \Box^{q_2} A_2$. This graded modal type is distinct from $U(F^q A)$ because the latter is a thunk which could contain arbitrary computations, whereas $\Box^q A$ merely scales a value. While it wouldn't solve the problem of imprecision, which arises out of multiplication—using 2 boxes of type $\Box^1 A$ would be imprecise if one of those uses ignored its payload—it would at least allow the components of a 1 pair, for instance, to be used in different quantities.

More precise static coeffect tracking can also arise out of a more expressive language of quantities. For instance, [Doré \[2026\]](#) embeds a quantitative type system in a dependent type theory such that the quantities are first-class terms that can depend on other terms. With such quantities, static tracking can be more precise in the presence of branching computations. A variable x might be used differently in two different branches, and its annotated quantity could depend on the scrutinee of the branch, thus tracking more precise quantities in each separate branch rather than approximating usage in both branches with a single quantity.

Dependent quantities affect the execution path as well, because they enable more opportunities for discarding unused values. (Recall that we have extended [Torczon et al.](#)'s theorem on 0-quantity variables to show that any 0-quantity value in a precise evaluation environment may be discarded.) A more precise type system could implement sound garbage collection algorithms that deallocate precisely when a resource is no longer used, rather than when it is no longer reachable.

For example, consider the following typing judgement.

$$\gamma \cdot \Gamma \vdash \text{let } x \leftarrow \text{return}^q V \text{ in case } y \text{ of } \text{inl } z \Rightarrow \text{return}^0 x \mid \text{inr } z \Rightarrow \text{return}^1 x : F^q A$$

This judgement could be derivable if q could depend on y , returning 0 for the left injection and 1 for the right. Static tracking in the body of the let expression would be precise for x , and no subsumption rule would be required. Then whether a reference to V is retained depends on whether q evaluates to 0. If it does, then V would be discarded for $\not\downarrow$ in rule [EX-RET-JUNK](#); otherwise, it would be evaluated by rule [EX-RET](#).

9 Conclusion

In this work, we have instrumented a precise coeffect-aware semantics for call-by-push-value, and shown it sound with respect to the resource-tracking type-and-coeffect system by [Torczon et al. \[2024\]](#). In particular, we have shown that well-typed programs have a valid evaluation with a *unique* precise environment. This means that our semantics is indeed precise because there cannot exist more precise coeffect annotations.

We have also shown that all raw evaluations have a corresponding (unique) precise evaluation environment when provided a desired annotation of the evaluation result, and that erasing those annotations once again yields the original raw evaluation. In other words, precise instrumentation only increases information without interfering with evaluation itself. Consequently, it would be possible to design even more precise type-and-coeffect systems on top of CBPV with coeffects and prove the existing precise semantics sound with respect to the new system rather than redesigning a coeffect-aware semantics that is tightly coupled to the type-and-coeffect system.

While we have instrumented precise semantics for CBPV with coeffects, the technique is not tied to CBPV and could be applied to other semantics by using our key insight of tracking coeffects on values rather than on variables during execution.

Acknowledgments

Thanks to Dominic Orchard for helpful discussion.

References

- Andreas Abel and Jean-Philippe Bernardy. 2020. A Unified View of Modalities in Type Systems. *Proc. ACM Program. Lang.* 4, ICFP, Article 90 (Aug. 2020), 28 pages. doi:10.1145/3408972
- Andreas Abel, Nils Anders Danielsson, and Oskar Eriksson. 2023. A Graded Modal Dependent Type Theory with a Universe and Erasure, Formalized. *Proc. ACM Program. Lang.* 7, ICFP, Article 220 (Aug. 2023), 35 pages. doi:10.1145/3607862
- Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (Oxford, United Kingdom) (LICS '18)*. Association for Computing Machinery, New York, NY, USA, 56–65. doi:10.1145/3209108.3209189
- Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. 2018. Linear Haskell: practical linearity in a higher-order polymorphic language. *Proc. ACM Program. Lang.* 2, POPL, Article 5 (Jan. 2018), 29 pages. doi:10.1145/3158093
- Riccardo Bianchini, Francesco Dagnino, Paola Giannini, and Elena Zucca. 2023. Resource-Aware Soundness for Big-Step Semantics. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 267 (Oct. 2023), 29 pages. doi:10.1145/3622843
- Riccardo Bianchini, Francesco Dagnino, Paola Giannini, and Elena Zucca. 2026. Don't exhaust, don't waste: Resource-aware soundness for big-step semantics. *Journal of Functional Programming* 36, Article 10 (June 2026), 54 pages. doi:10.46298/jfp.17800
- Aloïs Brunel, Marco Gaboardi, Damiano Mazza, and Steve Zdancewic. 2014. A Core Quantitative Coeffect Calculus. In *Programming Languages and Systems*, Zhong Shao (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 351–370. doi:10.1007/978-3-642-54833-8_19
- Jonathan Chan and Stephanie Weirich. 2026. *Artifact for "Resource Soundness with Precise Coeffect-Aware Semantics"*. doi:10.5281/zenodo.21330476
- Pritam Choudhury, Harley Eades III, Richard A. Eisenberg, and Stephanie Weirich. 2021. A Graded Dependent Type System with a Usage-Aware Semantics. *Proc. ACM Program. Lang.* 5, POPL, Article 50 (Jan. 2021), 32 pages. doi:10.1145/3434331
- Ugo Dal Lago and Francesco Gavazzo. 2022. A Relational Theory of Effects and Coeffects. *Proc. ACM Program. Lang.* 6, POPL, Article 31 (Jan. 2022), 28 pages. doi:10.1145/3498692
- Maximilian Doré. 2026. Dependent Multiplicities in Dependent Linear Type Theory. arXiv:2507.08759 [cs.PL] doi:10.48550/arXiv.2507.08759
- Oskar Eriksson, Nils Anders Danielsson, and Andreas Abel. 2026. On Recursion in Graded Modal Type Theory. *Proc. ACM Program. Lang.* 10, ICFP (Aug. 2026), 25 pages. To appear.
- Marco Gaboardi, Shin-ya Katsumata, Dominic Orchard, Flavien Breuvar, and Tarmo Uustalu. 2016. Combining Effects and Coeffects via Grading. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming (Nara, Japan) (ICFP 2016)*. Association for Computing Machinery, New York, NY, USA, 476–489. doi:10.1145/2951913.2951939
- Dan R. Ghica and Alex I. Smith. 2014. Bounded Linear Types in a Resource Semiring. In *Programming Languages and Systems*, Zhong Shao (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 331–350. doi:10.1007/978-3-642-54833-8_18
- Jonathan S. Golan. 2003. *Partially-ordered semimodules*. Springer Netherlands, Dordrecht, 137–149. doi:10.1007/978-94-017-0383-3_10
- Paul Blain Levy. 2003. *Call-By-Push-Value: A Functional/Imperative Synthesis*. Springer Dordrecht, Dordrecht, Netherlands. doi:10.1007/978-94-007-0954-6
- Vilem-Benjamin Liepelt, Danielle Marshall, Dominic Orchard, Vineet Rajani, and Michael Vollmer. 2026. *On Graded Coeffect Types for Information-Flow Control*. Springer Nature Switzerland, Cham, 114–148. doi:10.1007/978-3-032-08187-2_7
- Anton Lorenzen, Wenhao Tang, and Sam Lindley. 2026. A Value Trick for Modal Type Systems. Presentation at 8th Intuitionistic Modal Logic and Applications Workshop, Lisbon, Portugal. 20 pages. https://antonlorenzen.de/papers/value-trick-imla.pdf
- Anton Lorenzen, Leo White, Stephen Dolan, Richard A. Eisenberg, and Sam Lindley. 2024. Oxidizing OCaml with Modal Memory Management. *Proceedings of the ACM on Programming Languages* 8 (15 Aug. 2024), 485–514. Issue ICFP. doi:10.1145/3674642
- Klara Mazurak. 2013. *Linear Types, Protocols, and Processes in Classical F^o*. Ph. D. Dissertation. University of Pennsylvania, Philadelphia, PA, USA.

- Conor McBride. 2016. *I Got Plenty o' Nuttin'*. Springer International Publishing, Cham, 207–233. doi:10.1007/978-3-319-30936-1_12
- Benjamin Moon, Harley Eades III, and Dominic Orchard. 2021. Graded Modal Dependent Type Theory. In *Programming Languages and Systems*, Nobuko Yoshida (Ed.). Springer International Publishing, Cham, 462–490. doi:10.1007/978-3-030-72019-3_17
- Dominic Orchard, Vilem-Benjamin Liepelt, and Harley Eades III. 2019. Quantitative Program Reasoning with Graded Modal Types. *Proc. ACM Program. Lang.* 3, ICFP, Article 110 (July 2019), 30 pages. doi:10.1145/3341714
- Tomas Petricek, Dominic Orchard, and Alan Mycroft. 2013. Coeffects: Unified Static Analysis of Context-Dependence. In *Automata, Languages, and Programming*, Fedor V. Fomin, Rūsiņš Freivalds, Marta Kwiatkowska, and David Peleg (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 385–397. doi:10.1007/978-3-642-39212-2_35
- Tomas Petricek, Dominic Orchard, and Alan Mycroft. 2014. Coeffects: A Calculus of Context-Dependent Computation. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming* (Gothenburg, Sweden) (ICFP '14). Association for Computing Machinery, New York, NY, USA, 123–135. doi:10.1145/2628136.2628160
- Jason Reed and Benjamin C. Pierce. 2010. Distance Makes the Types Grow Stronger: A Calculus for Differential Privacy. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming* (Baltimore, Maryland, USA) (ICFP '10). Association for Computing Machinery, New York, NY, USA, 157–168. doi:10.1145/1863543.1863568
- Ilya Sergey, Dimitrios Vytiniotis, and Simon Peyton Jones. 2014. Modular, Higher-Order Cardinality Analysis in Theory and Practice. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (POPL '14). Association for Computing Machinery, New York, NY, USA, 335–347. doi:10.1145/2535838.2535861
- Cassia Torczon, Emmanuel Suárez Acevedo, Shubh Agrawal, Joey Velez-Ginorio, and Stephanie Weirich. 2024. Effects and Coeffects in Call-By-Push-Value. *Proc. ACM Program. Lang.* 8, OOPSLA (Oct. 2024), 27 pages. doi:10.1145/3689750
- David N. Turner, Philip Wadler, and Christian Mossin. 1995. Once Upon a Type. In *Proceedings of the Seventh International Conference on Functional Programming Languages and Computer Architecture* (FPCA '95). Association for Computing Machinery, New York, NY, USA, 1–11. doi:10.1145/224164.224168

A Omitted Rules

$$\begin{array}{c}
 \text{T-INR} \\
 \frac{\gamma \cdot \Gamma \vdash V : A_2}{\gamma \cdot \Gamma \vdash \text{inr } V : A_1 + A_2} \\
 \\
 \text{T-FST} \\
 \frac{\gamma \cdot \Gamma \vdash M : B_1 \& B_2}{\gamma \cdot \Gamma \vdash \text{fst } M : B_1} \\
 \\
 \text{T-PROD} \\
 \frac{\gamma \cdot \Gamma \vdash M_1 : B_1 \quad \gamma \cdot \Gamma \vdash M_2 : B_2}{\gamma \cdot \Gamma \vdash \langle M_1, M_2 \rangle : B_1 \& B_2} \\
 \\
 \text{T-SND} \\
 \frac{\gamma \cdot \Gamma \vdash M : B_1 \& B_2}{\gamma \cdot \Gamma \vdash \text{snd } M : B_2}
 \end{array}$$

Fig. 9: Omitted typing rules

$$\begin{array}{c}
 \text{EV-INR} \\
 \frac{\gamma \cdot \rho \vdash_v V \Downarrow W}{\gamma \cdot \rho \vdash_v \text{inr } V \Downarrow \text{inr } W} \\
 \\
 \text{EV-FST} \\
 \frac{\gamma \cdot \rho \vdash_v M \Downarrow \gamma' \cdot \rho' ; \langle M_1, M_2 \rangle \quad \gamma' \cdot \rho' \vdash_v M_1 \Downarrow T}{\gamma \cdot \rho \vdash_v \text{fst } M \Downarrow T} \\
 \\
 \text{EV-PROD} \\
 \frac{}{\gamma \cdot \rho \vdash_v \langle M_1, M_2 \rangle \Downarrow \gamma \cdot \rho ; \langle M_1, M_2 \rangle} \\
 \\
 \text{EV-SND} \\
 \frac{\gamma \cdot \rho \vdash_v M \Downarrow \gamma' \cdot \rho' ; \langle M_1, M_2 \rangle \quad \gamma' \cdot \rho' \vdash_v M_2 \Downarrow T}{\gamma \cdot \rho \vdash_v \text{snd } M \Downarrow T} \\
 \\
 \text{EV-CASE-INR} \\
 \frac{q \leq 1 \quad \gamma_1 \cdot \rho \vdash_v V \Downarrow \text{inr } W \quad \gamma_2 \cdot \rho, x_2 \mapsto^q W \vdash_v M_2 \Downarrow T}{(q \cdot \gamma_1 + \gamma_2) \cdot \rho \vdash_v \text{case } V^q \text{ of } \text{inl } x_1 \Rightarrow M_1 \mid \text{inr } x_2 \Rightarrow M_2 \Downarrow T}
 \end{array}$$

Fig. 10: Omitted vector evaluation rules

$$\begin{array}{c}
 \text{EX-INR} \\
 \frac{\rho \vdash_P V \Downarrow W}{\rho \vdash_P \text{inr } V \Downarrow \text{inr}^q W} \\
 \\
 \text{EX-FST} \\
 \frac{\rho \vdash_P M \Downarrow \rho' ; \langle M_1, M_2 \rangle \quad \rho' \vdash_P M_1 \Downarrow T}{\rho \vdash_P \text{fst } M \Downarrow T} \\
 \\
 \text{EX-PROD} \\
 \frac{}{\rho \vdash_P \langle M_1, M_2 \rangle \Downarrow \rho ; \langle M_1, M_2 \rangle} \\
 \\
 \text{EX-SND} \\
 \frac{\rho \vdash_P M \Downarrow \rho' ; \langle M_1, M_2 \rangle \quad \rho' \vdash_P M_2 \Downarrow T}{\rho \vdash_P \text{snd } M \Downarrow T} \\
 \\
 \text{EX-CASE-INR} \\
 \frac{q \neq 0 \quad \rho_1 + \rho_2 = \rho \quad \rho_1 \vdash_P V \Downarrow \text{inr}^q W \quad \rho_2, x_2 \mapsto W \vdash_P M_2 \Downarrow T}{\rho \vdash_P \text{case } V^q \text{ of } \text{inl } x_1 \Rightarrow M_1 \mid \text{inr } x_2 \Rightarrow M_2 \Downarrow T}
 \end{array}$$

Fig. 11: Omitted precise evaluation rules

$$\begin{array}{c}
 \text{ADD-INR} \\
 \frac{W_1 + W_2 = W}{\text{inr}^{q_1} W_1 + \text{inr}^{q_2} W_2 = \text{inr}^{q_1+q_2} W} \\
 \\
 \text{SUB-INR} \\
 \frac{q_1 \leq q_2 \quad W_1 \leq W_2}{\text{inr}^{q_1} W_1 \leq \text{inr}^{q_2} W_2} \\
 \\
 \text{D-INR} \\
 \frac{W_1 \sqsubseteq W_2}{\text{inr}^q W_1 \sqsubseteq \text{inr}^q W_2}
 \end{array}$$

Fig. 12: Omitted adding, subsumption, and discarding rules

B Raw Semantics

$$\begin{aligned}
 W &::= () \mid \text{inl } W \mid \text{inr } W \mid (W_1, W_2) \mid \{\rho; M\} \mid \downarrow \\
 T &::= \text{return } W \mid \rho; \lambda x. M \mid \rho; \langle M_1, M_2 \rangle \\
 \rho &::= \emptyset \mid \rho, x \mapsto W
 \end{aligned}$$

$$\boxed{\rho \vdash_R V \Downarrow W}$$

(value evaluation)

E-VAR

$$\frac{}{\rho \vdash_R x \Downarrow \rho(x)}$$

E-UNIT

$$\frac{}{\rho \vdash_R () \Downarrow ()}$$

E-PAIR

$$\frac{\rho \vdash_R V_1 \Downarrow W_1 \quad \rho \vdash_R V_2 \Downarrow W_2}{\rho \vdash_R (V_1, V_2) \Downarrow (W_1, W_2)}$$

E-INL

$$\frac{\rho \vdash_R V \Downarrow W}{\rho \vdash_R \text{inl } V \Downarrow \text{inl } W}$$

E-INR

$$\frac{\rho \vdash_R V \Downarrow W}{\rho \vdash_R \text{inr } V \Downarrow \text{inr } W}$$

E-THUNK

$$\frac{}{\rho \vdash_R \{M\} \Downarrow \{\rho; M\}}$$

$$\boxed{\rho \vdash_R M \Downarrow T}$$

(computation evaluation)

E-APP

$$\frac{\begin{array}{c} q \neq 0 \\ \rho \vdash_R M \Downarrow \rho'; \lambda x. M' \\ \rho \vdash_R V \Downarrow W \\ \rho', x \mapsto W \vdash_R M' \Downarrow T \end{array}}{\rho \vdash_R M V^q \Downarrow T}$$

E-FORCE

$$\frac{\begin{array}{c} \rho \vdash_R V \Downarrow \{\rho'; M\} \\ \rho' \vdash_R M \Downarrow T \end{array}}{\rho \vdash_R V! \Downarrow T}$$

E-ABS

$$\frac{}{\rho \vdash_R \lambda x. M \Downarrow \rho; \lambda x. M}$$

E-RET

$$\frac{q \neq 0 \quad \rho \vdash_R V \Downarrow W}{\rho \vdash_R \text{return}^q V \Downarrow \text{return } W}$$

E-LET

$$\frac{\begin{array}{c} \rho \vdash_R M_1 \Downarrow \text{return } W \\ \rho, x \mapsto W \vdash_R M_2 \Downarrow T \end{array}}{\rho \vdash_R \text{let } x \leftarrow M_1 \text{ in } M_2 \Downarrow T}$$

E-PROD

$$\frac{}{\rho \vdash_R \langle M_1, M_2 \rangle \Downarrow \rho; \langle M_1, M_2 \rangle}$$

E-FST

$$\frac{\begin{array}{c} \rho \vdash_R M \Downarrow \rho'; \langle M_1, M_2 \rangle \\ \rho' \vdash_R M_1 \Downarrow T \end{array}}{\rho \vdash_R \text{fst } M \Downarrow T}$$

E-SND

$$\frac{\begin{array}{c} \rho \vdash_R M \Downarrow \rho'; \langle M_1, M_2 \rangle \\ \rho' \vdash_R M_2 \Downarrow T \end{array}}{\rho \vdash_R \text{snd } M \Downarrow T}$$

E-SEQ

$$\frac{\begin{array}{c} q \neq 0 \\ \rho \vdash_R V \Downarrow () \quad \rho \vdash_R M \Downarrow T \end{array}}{\rho \vdash_R V^q; M \Downarrow T}$$

E-MATCH

$$\frac{\begin{array}{c} q \neq 0 \quad \rho \vdash_R V \Downarrow (W_1, W_2) \\ \rho, x_1 \mapsto W_1, x_2 \mapsto W_2 \vdash_R M \Downarrow T \end{array}}{\rho \vdash_R \text{match } V^q \text{ as } (x_1, x_2) \Rightarrow M \Downarrow T}$$

E-CASE-INL

$$\frac{\begin{array}{c} q \neq 0 \quad \rho \vdash_R V \Downarrow \text{inl } W \\ \rho, x_1 \mapsto W \vdash_R M_1 \Downarrow T \end{array}}{\rho \vdash_R \text{case } V^q \text{ of } \text{inl } x_1 \Rightarrow M_1 \mid \text{inr } x_2 \Rightarrow M_2 \Downarrow T}$$

E-CASE-INR

$$\frac{\begin{array}{c} q \neq 0 \quad \rho \vdash_R V \Downarrow \text{inr } W \\ \rho, x_2 \mapsto W \vdash_R M_2 \Downarrow T \end{array}}{\rho \vdash_R \text{case } V^q \text{ of } \text{inl } x_1 \Rightarrow M_1 \mid \text{inr } x_2 \Rightarrow M_2 \Downarrow T}$$

E-APP-JUNK

$$\frac{\begin{array}{c} q = 0 \\ \rho \vdash_R M \Downarrow \rho'; \lambda x. M' \\ \rho', x \mapsto \downarrow \vdash_R M' \Downarrow T \end{array}}{\rho \vdash_R M V^q \Downarrow T}$$

E-RET-JUNK

$$\frac{q = 0}{\rho \vdash_R \text{return}^q V \Downarrow \text{return } \downarrow}$$

E-SEQ-JUNK

$$\frac{q = 0 \quad \rho \vdash_R M \Downarrow T}{\rho \vdash_R V^q; M \Downarrow T}$$

E-MATCH-JUNK

$$\frac{\begin{array}{c} q = 0 \\ \rho, x_1 \mapsto \downarrow, x_2 \mapsto \downarrow \vdash_R M \Downarrow T \end{array}}{\rho \vdash_R \text{match } V^q \text{ as } (x_1, x_2) \Rightarrow M \Downarrow T}$$

Fig. 13: Big-step raw resource semantics [cbpv.raw.semantics:EvalVal, EvalComp]

C Vector Annotation of Raw Semantics

PROPOSITION C.1 (VECTOR ANNOTATION OF RAW SEMANTICS).

- Suppose $\rho \vdash_R V \Downarrow W$. Then for any W' such that $|W'| = W$, there exist γ, ρ' such that $\gamma \cdot \rho' \vdash_V V \Downarrow W'$, where $|\rho'| = \rho$.
- Suppose $\rho \vdash_R M \Downarrow T$. Then for any T' such that $|T'| = T$, there exist γ, ρ' such that $\gamma \cdot \rho' \vdash_V M \Downarrow T'$, where $|\rho'| = \rho$.

Proof attempt sketch. By induction on the derivation of raw evaluation. As an example, consider the case of rule **E-PAIR**, along with some (W'_1, W'_2) such that $|W'_1| = W_1$ and $|W'_2| = W_2$.

$$\frac{\rho \vdash_R V_1 \Downarrow W_1 \quad \rho \vdash_R V_2 \Downarrow W_2}{\rho \vdash_R (V_1, V_2) \Downarrow (W_1, W_2)}$$

By the induction hypotheses, we have:

- γ_1, ρ'_1 such that $\gamma_1 \cdot \rho'_1 \vdash_V V_1 \Downarrow W'_1$ and $|\rho'_1| = \rho$; and
- γ_2, ρ'_2 such that $\gamma_2 \cdot \rho'_2 \vdash_V V_2 \Downarrow W'_2$ and $|\rho'_2| = \rho$.

The goal is to show that $(\gamma_1 + \gamma_2) \cdot \rho' \vdash_V (V_1, V_2) \Downarrow (W'_1, W'_2)$ for some ρ' such that $|\rho'| = \rho$. However, using rule **Ev-PAIR**, we require $\gamma_1 \cdot \rho' \vdash_V V_1 \Downarrow W'_1$ and $\gamma_2 \cdot \rho' \vdash_V V_2 \Downarrow W'_2$, while we have no guarantee that $\rho'_1 = \rho'_2$. $\square$

As a concrete counterexample, the raw evaluation

$$x \mapsto () \vdash_R \text{let } y \leftarrow \text{return}^1 x \text{ in } y; \text{return}^1 y \Downarrow \text{return}()$$

cannot be annotated into a vector evaluation, since y is used twice while $\text{return}^1 x$ has the wrong annotation. This suggests that raw return computations shouldn't have quantity annotations at all (aside perhaps from indicating whether a junk rule should be used or not). However, if they aren't annotated, then the following raw evaluation

$$x \mapsto \{y \mapsto W; \text{return } y\} \vdash_R x \Downarrow \{y \mapsto W; \text{return } y\}$$

may be annotated with an arbitrary q as the vector evaluation

$$x \mapsto^1 \{y \mapsto^q W; \text{return}^q y\} \vdash_V x \Downarrow \{y \mapsto^q W; \text{return}^q y\}$$

meaning that when evaluating (x, x) , the induction hypotheses in the proof attempt above definitely could yield incompatible vector environments by using two different qs .

D Logical Relation

$\boxed{\mathcal{W}[A]}$ $\boxed{\mathcal{T}[B]}$ (interpretation of types as sets of results)

$$\mathcal{W}[\top] = \{()\}$$

$$\mathcal{W}[A_1 + A_2] = \{\text{inl } W \mid W \in \mathcal{W}[A_1]\} \cup \{\text{inr } W \mid W \in \mathcal{W}[A_2]\}$$

$$\mathcal{W}[A_1 \times A_2] = \{(W_1, W_2) \mid W_1 \in \mathcal{W}[A_1] \wedge W_2 \in \mathcal{W}[A_2]\}$$

$$\mathcal{W}[\cup B] = \{\{\gamma \cdot \rho; M\} \mid \gamma \cdot \rho \models M \in \mathcal{M}[B]\}$$

$$\mathcal{T}[F^q A] = \begin{cases} \{\text{return}^q W \mid W \in \mathcal{W}[A]\} & \text{if } q \neq 0 \\ \{\text{return}^q \zeta\} & \text{if } q = 0 \end{cases}$$

$$\mathcal{T}[A^q \rightarrow B] = \begin{cases} \{\{\gamma \cdot \rho; \lambda x. M \mid \forall W \in \mathcal{W}[A], \gamma \cdot \rho, x \mapsto^q W \models M \in \mathcal{M}[B]\} & \text{if } q \neq 0 \\ \{\{\gamma \cdot \rho; \lambda x. M \mid \gamma \cdot \rho, x \mapsto^q \zeta \models M \in \mathcal{M}[B]\} & \text{if } q = 0 \end{cases}$$

$$\mathcal{T}[B_1 \& B_2] = \{\{\gamma \cdot \rho; \langle M_1, M_2 \rangle \mid \gamma \cdot \rho \models M_1 \in \mathcal{M}[B_1] \wedge \gamma \cdot \rho \models M_2 \in \mathcal{M}[B_2]\}$$

Fig. 14: Logical relation over value and computation types [cbpv.vector.semtyping:LRV,LRC]